



Requirement Analysis Specification Document (RASD)

Software Engineering 2 Project, 2019/2020

Version 2.0

Studente	Matricola
Riccardo Colella	944961
Matteo Ferrini	944759
Francesco Gianchino	946512

Table of contents

Requirement Analysis Specification Document (RASD)	0
Table of contents	1
1. Introduction	3
1.1 Purpose	3
1.1.1 General purpose	3
1.1.2 Goals	4
1.2 Scope	4
1.3 Definitions, acronyms, abbreviations	6
1.3.1 Definitions	6
1.3.2 Acronyms	6
1.3.3 Abbreviations	6
1.4 References	7
1.5 Document Overview	7
1.6 Revision History	7
2. Overall description	9
2.1 Product perspective	9
2.2 Product functions	15
2.2.1 Report violation	15
2.2.2 Mining informations	15
2.2.3 Suggest interventions	15
2.2.4 Generation of traffic tickets	16
2.2.5 Statistics	16
2.3 User characteristics	16
2.4 Assumptions, dependencies and constraints	17
3. Specific requirements	18
3.1 External interfaces requirements	18
3.1.1 User interfaces	18
3.1.2 Hardware interfaces	21
3.1.3 Software interfaces	21
3.1.4 Communication interfaces	21
3.2 Functional requirements	21
3.2.1 User	21
3.1.2 Authority	32
3.2.3 Requirements	45
3.2.4 Traceability matrix	49
3.3 Performance requirements	50
3.4 Design constraints	50
3.4.1 Standard compliance	50

	2
3.4.2 Hardware limitations	50
3.4.3 Any other constraint	50
3.5 Software system attributes	51
3.5.1 Reliability	51
3.5.2 Availability	51
3.5.3 Security	51
3.5.4 Maintainability	51
3.5.5 Portability	51
4. Formal analysis using Alloy	52
5. Effort spent	64
6. References	66

1. Introduction

1.1 Purpose

1.1.1 General purpose

SafeStreets is a crowd-sourced application that lets users notify authorities when traffic violations occur. It offers a basic service and two advanced functions. The service can be used upon registration.

Basic service

SafeStreets collects the data (photo, date, time, position and type of the violation) provided by the user, completes it with suitable metadata, processes the picture through an image recognition algorithm and eventually stores the data in its servers (if no errors occurred).

Furthermore, SafeStreets offers its users the possibility to mine the collected information, providing different levels of visibility based on the role of the applicant (citizen or authority).

Advanced Function 1

SafeStreets crosses its own data with those about accidents provided by municipalities in order to identify potentially risky areas and suggest possible interventions.

Advanced Function 2

Given the fact that municipalities could use SafeStreets information to issue violation tickets, the application implements a mechanism which ensures data to be always valid, never damaged nor altered. Additionally, SafeStreets uses the

information about issued tickets to build statistics or to strengthen the validity of its initiative.

1.1.2 Goals

These are the application goals:

- **G01.** The application must allow the user to send photos about parking violations.
- **G02.** The application must allow the user to consult his own data.
- **G03.** The application must allow the user and authorities to extract the data about about parking violations.
- **G04.** The application must provide different levels of visibility of the requested data based on the role of the applicant.
- **G05.** The application must be capable of retrieving information from municipality (those offering the services) about accidents.
- **G06.** The application must provide the municipality suggestions about possible interventions.
- **G07.** The application must allow the municipality to retrieve the information about the violation to generate traffic tickets.
- **G08.** The application must be capable of retrieving information from municipality about issued tickets.
- **G09.** The application must allow the authority to see stats about the efficiency of the service.

1.2 Scope

SafeStreets intends to provide users with the possibility to notify authorities when traffic violations occur and in particular parking violations. It also aims to help authorities (generic) in improving their penalty system and get information about correlations in violations and city's areas.

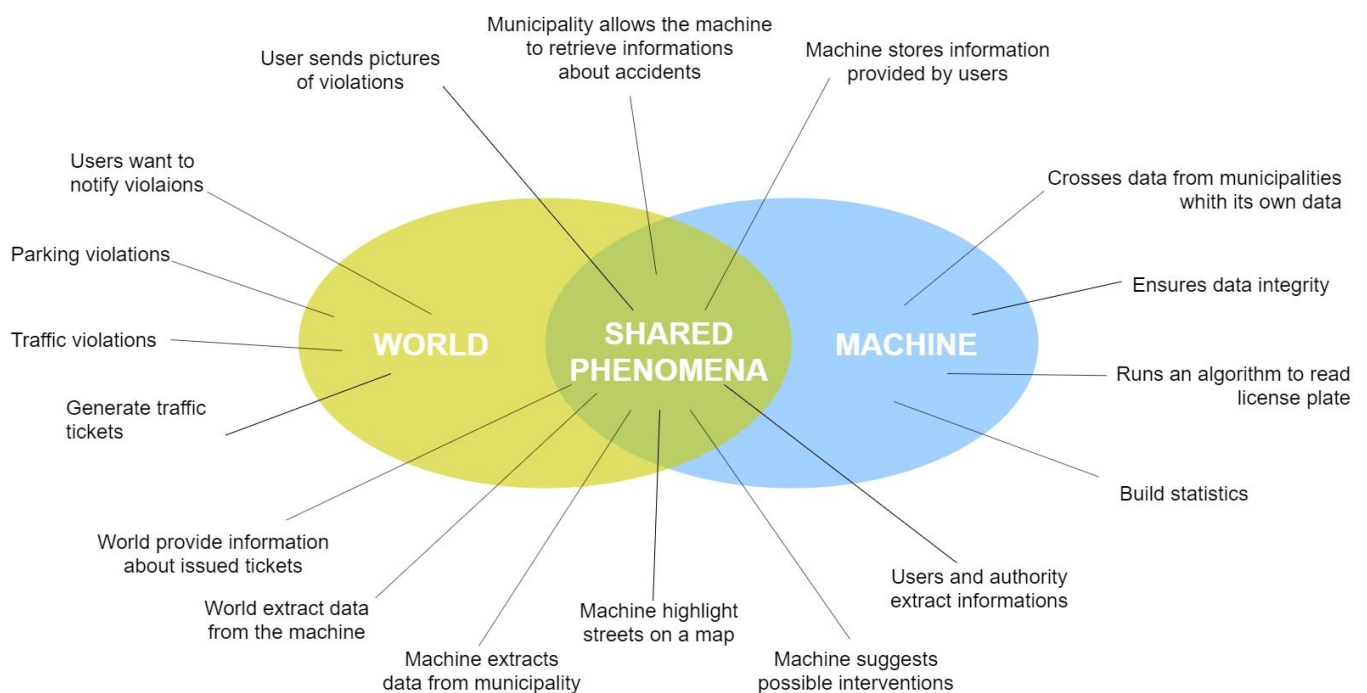
Given that, we can divide every system into two parts:

- The **machine**: the portion of system to be developed.
- The **world**: the portion of the real-world affected by the machine. This part is commonly known as the *environment*.

As a consequence we can classify phenomena in three different types:

- **World phenomena**: phenomena that the machine cannot observe.
- **Machine phenomena**: phenomena located entirely in the machine.
- **Shared phenomena**: phenomena that can be controlled by the world and observed by the machine or controlled by the machine and observed by the world.

Here it is an analysis about the world, the machine and their phenomena.



1.3 Definitions, acronyms, abbreviations

1.3.1 Definitions

- **Applicant:** we define applicant as the entity who makes the requests to extract the data about parking violations.
- **Authority:** it is the service's consumer which has to grant the safety of an area. For more details see section [2.3 User characteristics](#).
- **Municipality:** it is a particular case of Authority. It can access a broader set of functionalities than a normal authority.
- **SafeStreets:** it is the name of the all software described by this paper. It includes different functions.
- **User:** it is the common single consumer of the service. For more details see section [2.3 User characteristics](#).

1.3.2 Acronyms

- **API:** Application Programming Interface
- **DB:** Database
- **GPS:** Global Positioning System
- **GDPR:** General Data Protection Regulation
- **HW:** Hardware
- **RASD:** Requirement Analysis Specification Document
- **UML:** Unified modelling language

1.3.3 Abbreviations

- **[Gxx]:** goal number xx
- **[Rxx]:** requirement number xx

- **[Dxx]**: domain assumption number xx
- **[U.C.x]**: use case number x
- **App**: application
- **Stats**: statistics

1.4 References

- Specification document “SafeStreets Mandatory Project Assignment”.

1.5 Document Overview

This document consists in the following structure:

Chapter 1 contains a brief introduction to the application. Furthermore it lists the goals which the application has to reach, the scope of the project and some more information needed in order to understand the application.

Chapter 2 contains an overall description of the product. Here are explained product functions and perspective. Here is also explained the functioning of the program, thanks to some descriptive diagrams that describe the major functions of the software and the assumptions under which it works as explained.

Chapter 3 aims to explain all the requirements needed so that every goal can be pursued given the situation described by the domain assumptions. Here are also presented some scenarios that describe some use cases. The use cases are defined through some explicative diagrams and describe all possible limitations due to hardware or software.

Chapter 4 contains a modelling of the project through the Alloy language.

Chapter 5 lists the number of hours spent by each member on the project.

1.6 Revision History

- **10/11/19 - Version 1.0**: First release.

- **27/11/19 - Version 1.1:** Correction of some oversights, addition of ux diagrams.
- **29/11/19 - Version 1.2:** Format adjustments, inserted diagram lost in previous versions.
- **15/12/19 - Version 2.0:** Final release.

2. Overall description

2.1 Product perspective

The high-level class diagram below provides a model of the application domain: it contains only few essential attributes for the various classes and does not include every class that will be necessary to define the Model (useful data) of the system.

The municipality is an extension of the authority and unlike the latter also contains data on accidents that occurred in streets, suggestions for managing these incidents and the tickets generated.

In “Picture of Violation” class a Boolean has been included indicating whether the photo is manipulated (or illegible) or not.

The information mined by user and authority (including Municipality) are lists of "InformationDataMining", each of the elements of these lists contains the name of the street where at least one report was made by a user.

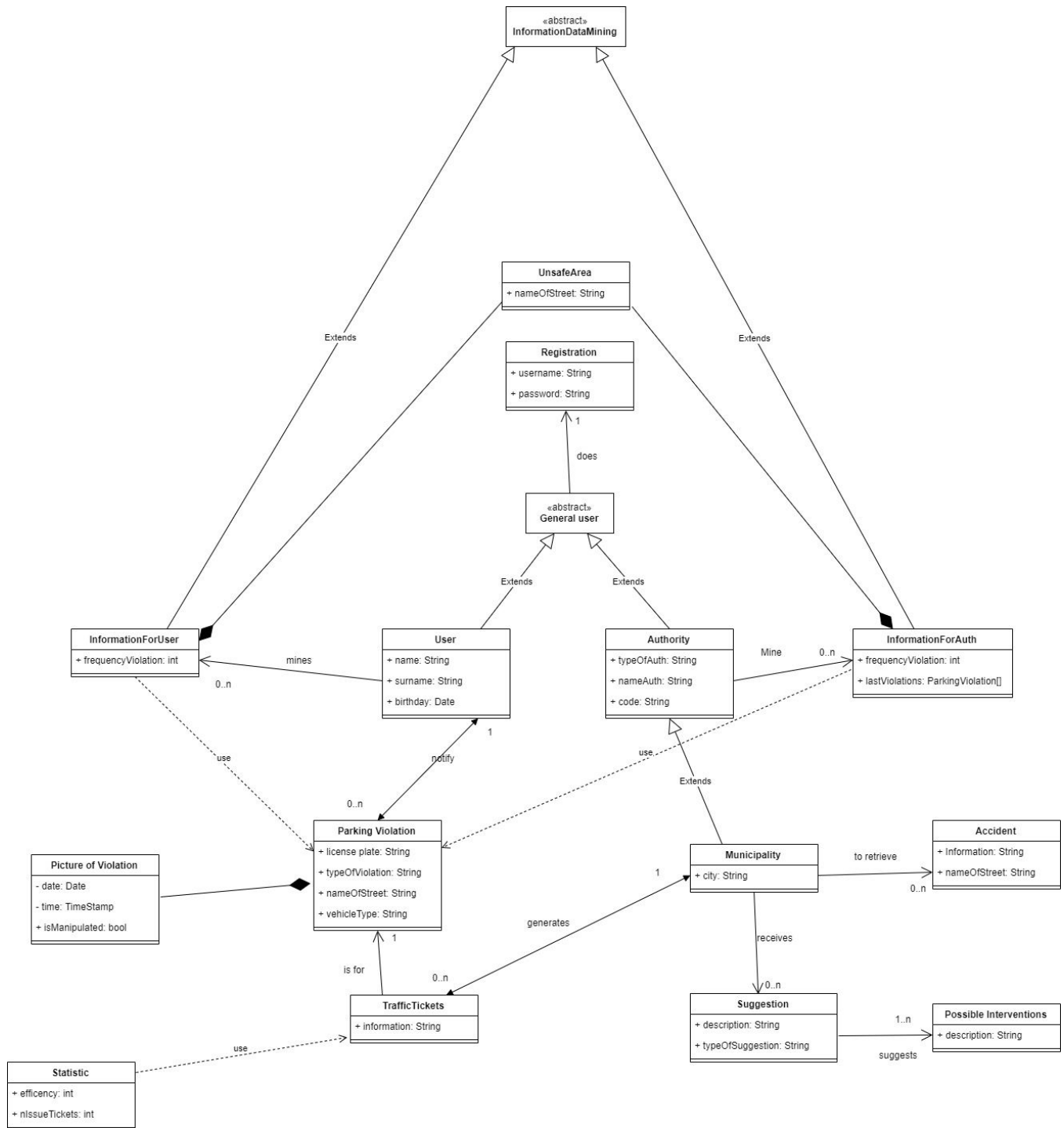
While the information mined by the users for each area (street) only contains the frequency of violations for the authorities, there is also complete information on the most recent parking violations made by users in that area.

The authorities are identified by the type and an identification code while the users are identified by the classic: name, surname and date of birth.

The Municipality has also the name of the city where it is located.

The statistics are generated by safestreets using the tickets data generated by the municipalities.

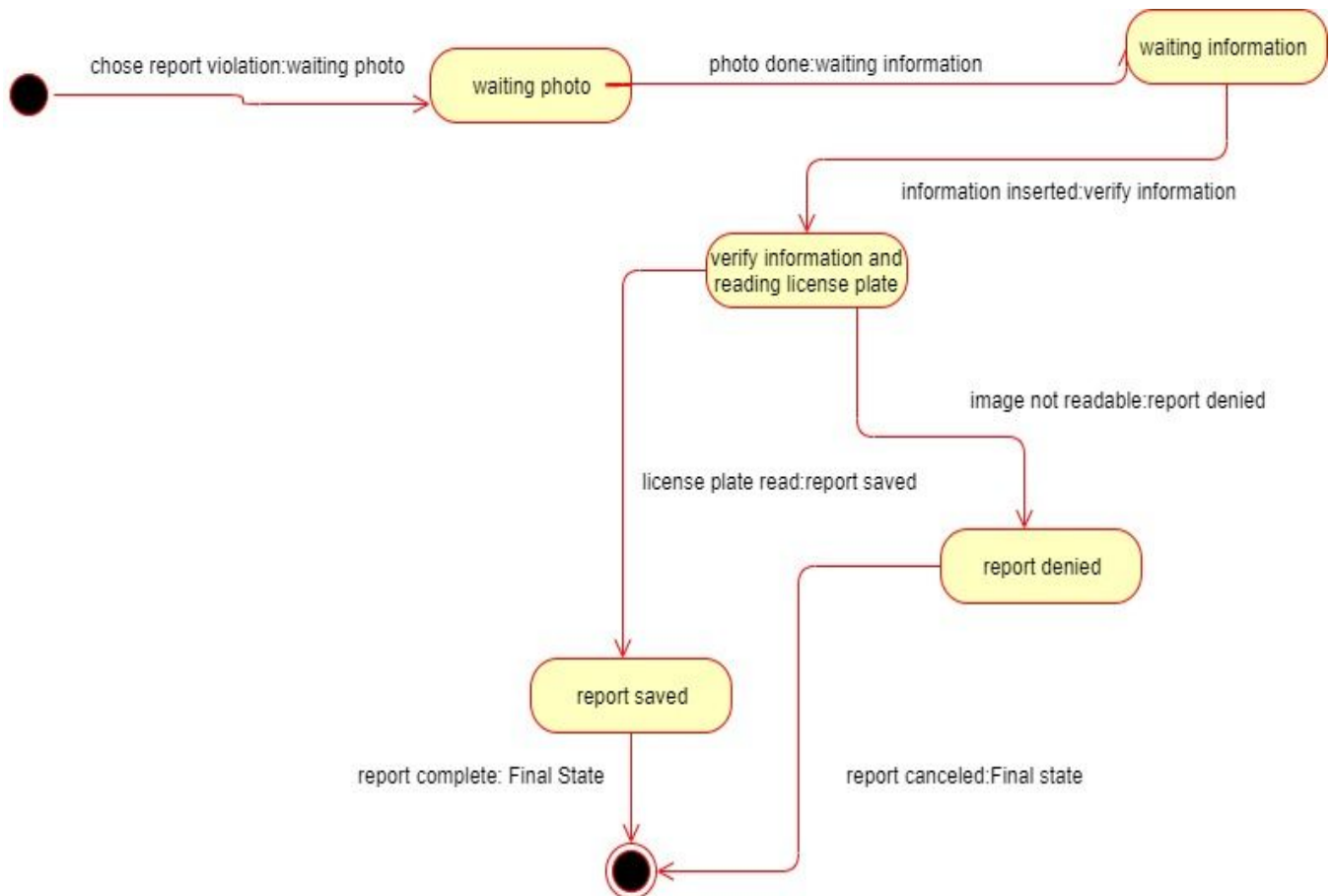
A parking violation is associated with each ticket generated by Municipality.



The state diagrams are reported below, they model the behaviour and show the conducts of some aspects of the application although not in a detailed way.

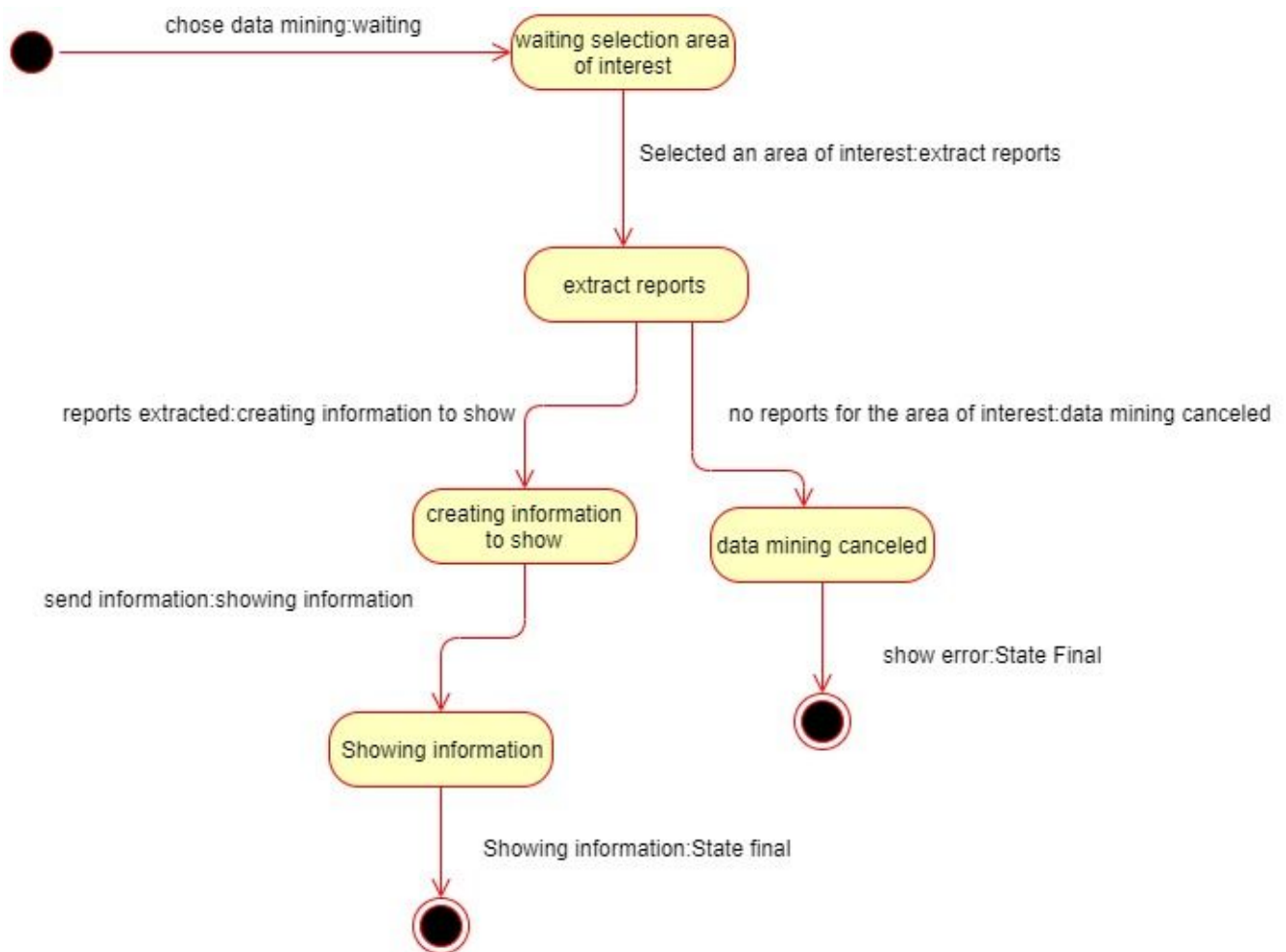
State Diagram 1: Report a Violation

In this diagram it is modelled the creation and the sending of a report to point out a violation. If the user's image is manipulated or not readable or the license plate can't be obtained, then the report is discarded.



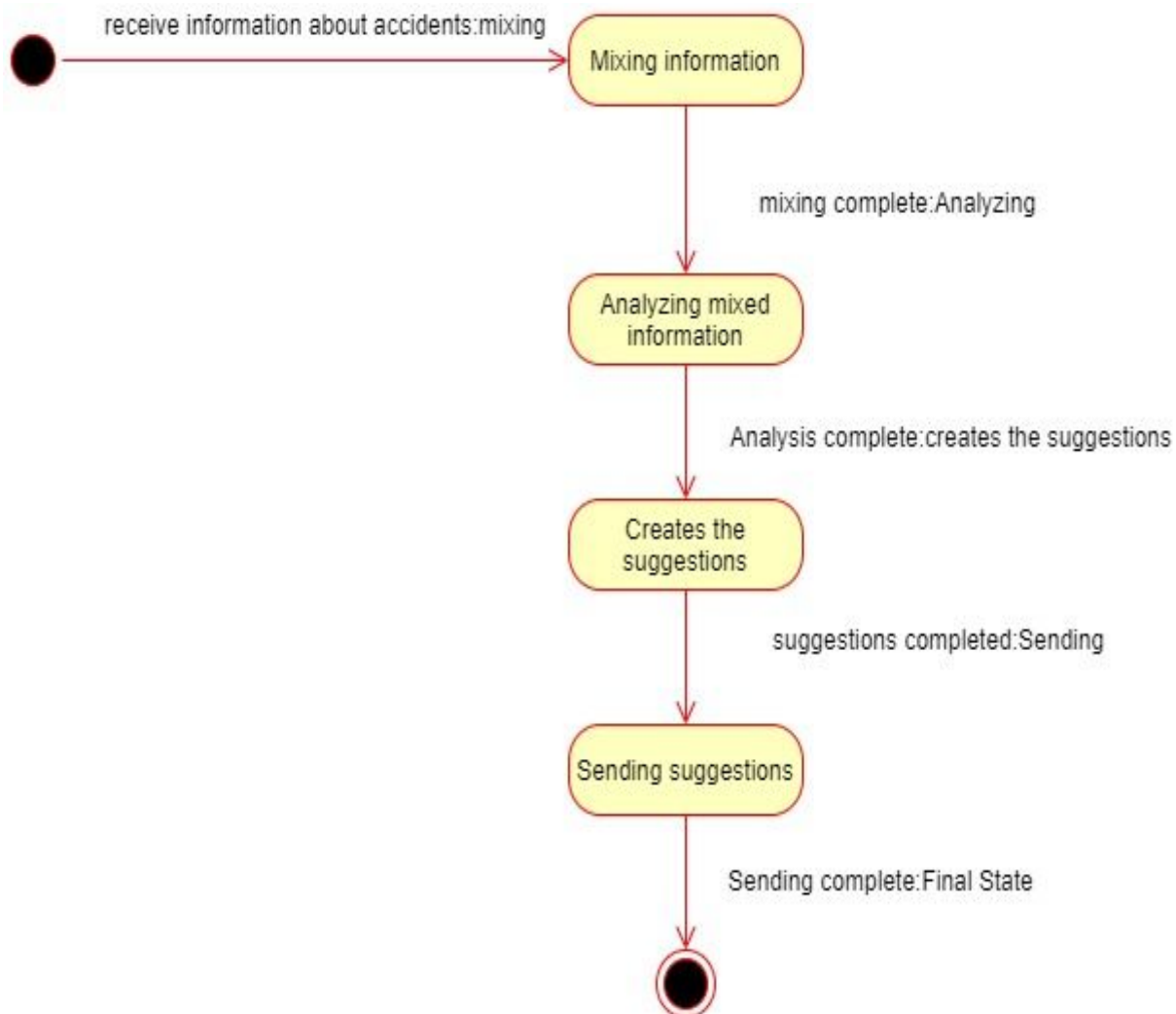
State Diagram 2: Mining information

In this diagram it is modelled the request to extract the data of the reports concerning a specific area. After that the applicant selects an area and the application extracts the informations and selects what to show. The selection depends on the applicant, whether it is a user or an authority.



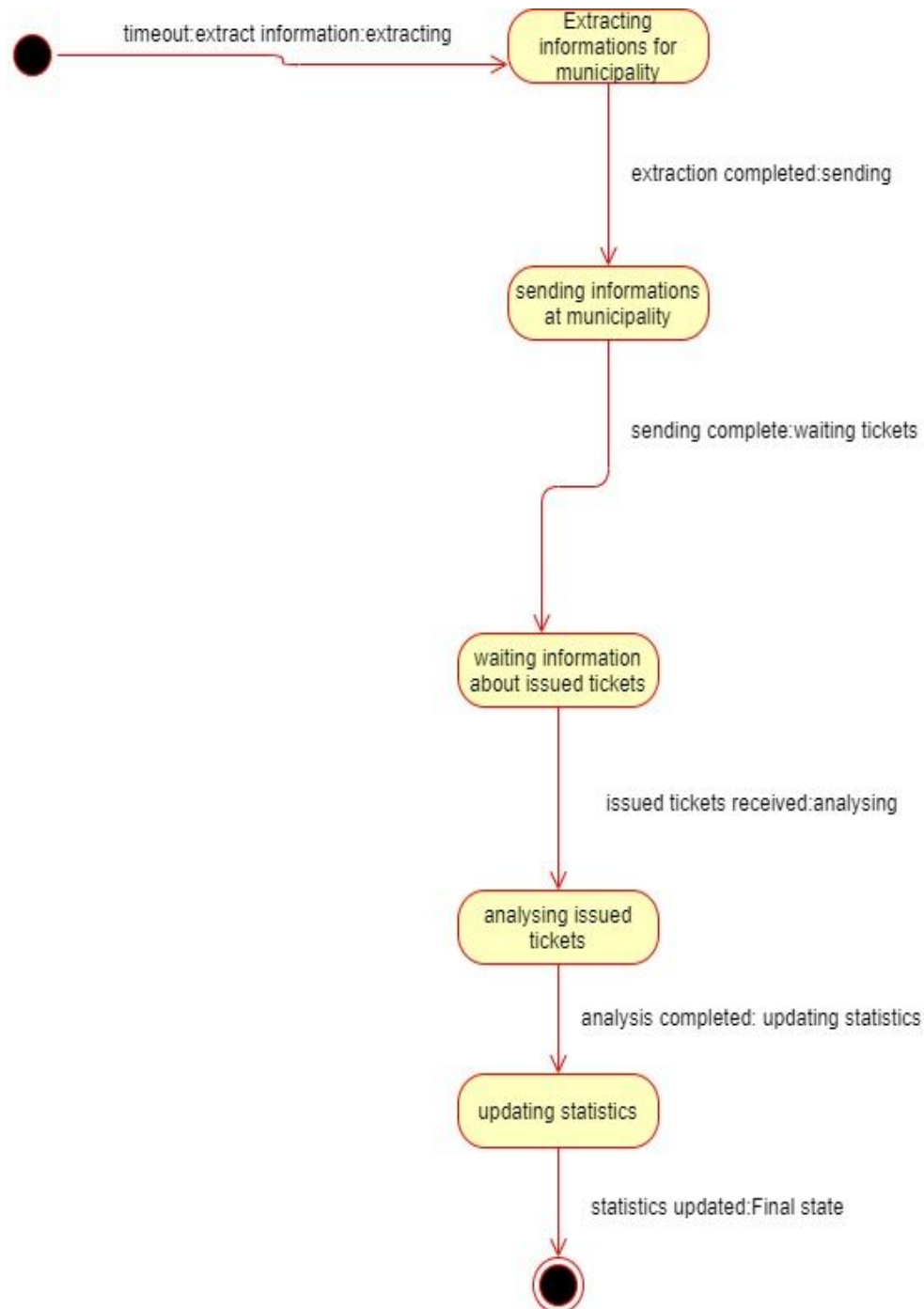
State Diagram 3: Mixing information and Create Suggestions

This diagram models the receiving of the information about accidents from municipality. The application after that has received the information, mixes it with its own, creates the suggestions for the area of accidents and send them to the municipality.



State Diagram 4: Sharing reports, analyzing issued tickets and update stats.

In this diagram is modelled the sharing of the reports with the municipality. The municipality obtains the reports, creates the tickets and after that sends the information about the tickets to the application. This information is used to update the statistics about the efficiency of the service.



2.2 Product functions

2.2.1 Report violation

As a crowd sourced application *SafeStreets* aims to allow user to notify authority when traffic violations occur, in particular parking violations. The user can send pictures of the violation that will include extra information like time, data and type of violation. An example of a violation is a bike parked in the middle of bike lanes, or in places reserved for people with disabilities. The report is then added with some additional information provided by *SafeStreet*: license plate is read by an algorithm and the gps position is retrieved using the smartphone's hardware.

2.2.2 Mining informations

All the reports sent via *SafeStreets* are stored in an internal database. Every user can access his segnalations and see statistics about aggregated data in a specific area. For example an user can ask for the city of Milan and see an highlighted map depending on the frequency of violations in a specific street. Another information that can be mined are the vehicles that committed the most violations. Of course different levels of visibility are offered to different roles.

2.2.3 Suggest interventions

Depending on the fact that the municipality has to offer a service that allows users to retrieve the information about the accidents that occur on the territory of the municipality, *SafeStreets* can offer another service. If the municipality shares its data, *SafeStreets* can cross that information with its own data to identify potentially unsafe areas and suggest possible interventions. For example: *SafeStreets* can cross data about the segnalations of illegal parkings and the number of accidents (with those cars involved) in a street and detect a problem of maximum speed limit in that area. Thus the application can suggest the municipality to add some bumps in order to control the car's speed.

2.2.4 Generation of traffic tickets

SafeStreets can also provide a service that allows the municipality to use the information coming from the application and generate automatically traffic tickets. In this case a mechanism to avoid forgery is put in place. *SafeStreets* will use software for analysing images and check that all the photo are shot in the correct time, date and position and the images are not manually altered. License plates will be recognised automatically.

2.2.5 Statistics

SafeStreets can use information about generated traffic tickets to create statistics. The most egregious offenders could be discovered and the service could be evaluated measuring issuing trends in response of the generated tickets.

2.3 User characteristics

The system aims at creating a service that can be useful for two types of consumers:

- The **user** is an individual person who wants to exploit the service offered by the system. He may wants to report a parking violations or to access to aggregate data in order to discover some interesting informations, such as the worst areas in terms of infractions.
- The **authority** is the municipality or the body that must grant security on public areas. This body can mine data about user-generated reports and access statistics about violations. Furthermore it can be extended into a *Municipality*, an entity which may want to automatize traffic tickets creation, that can obtain suggestions and shares (if qualified) info about accidents and traffic tickets with *SafeStreets*.

2.4 Assumptions, dependencies and constraints

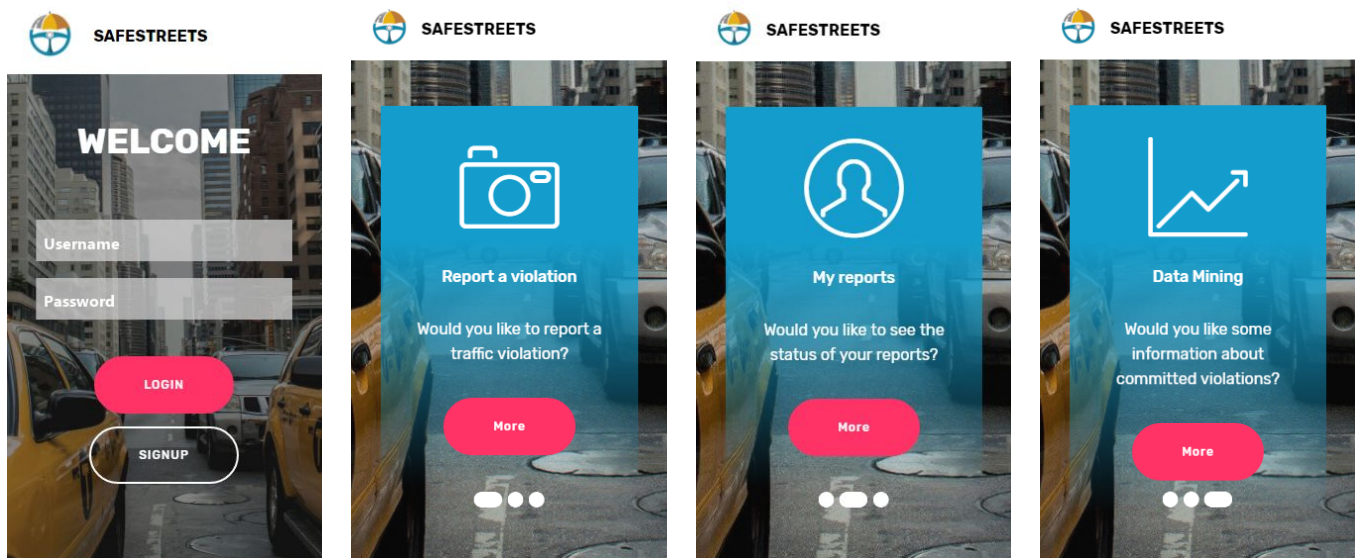
Domain assumptions help to make clear what it is expected from the external environment and allow constraining the software-driven machine so as to stay in the domain of interest.

- **D01:** Each license plate is a unique identifier.
- **D02:** The devices that acquires violations' position provides location with an error of 5 meters at most.
- **D03:** The internet connection works properly without failure.
- **D04:** The information about the accidents of the municipality are correct.
- **D05:** The authorities and in particular the municipality have demonstrated that they are acknowledged public institutions.
- **D06:** Municipality is a particular case of authority.
- **D07:** The information coming from the users is never broken if the image is not manipulated and the license plate is readable.
- **D08:** Each municipality that shares information with *SafeStreets* has authorized the app to take the informations that has inside.
- **D09:** The data and the time is set correctly on the device of the user and authorities.
- **D10:** Users can correctly identify the type of violation.

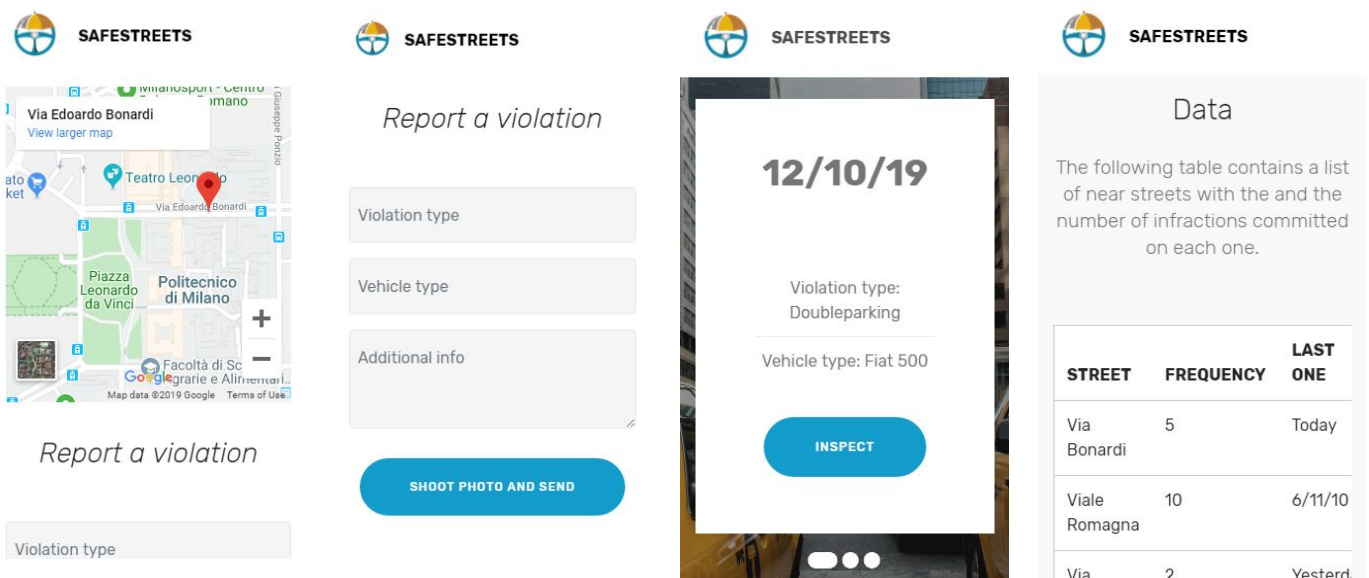
3. Specific requirements

3.1 External interfaces requirements

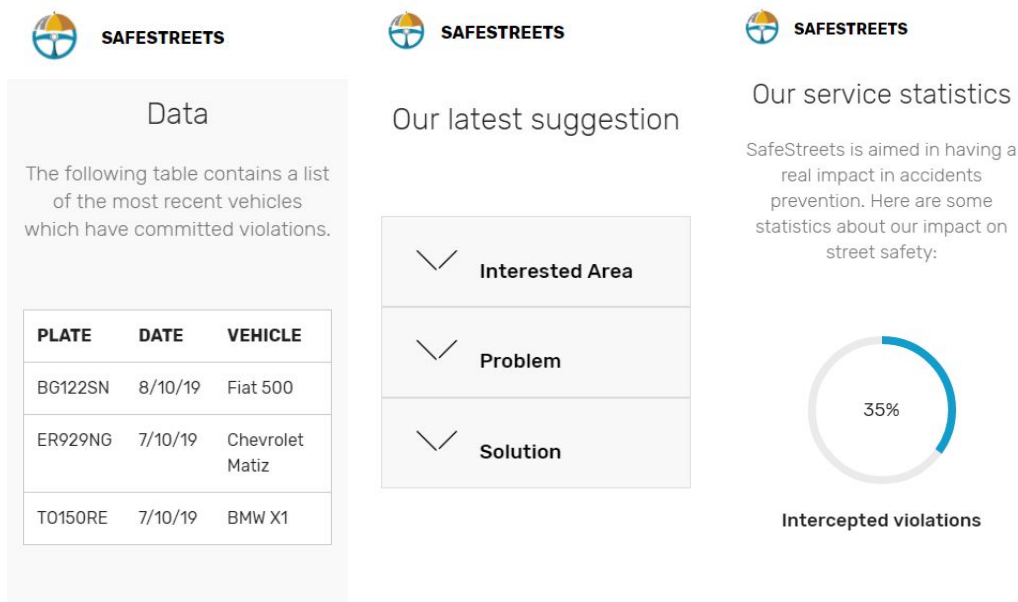
3.1.1 User interfaces



Login and main menu for the user.

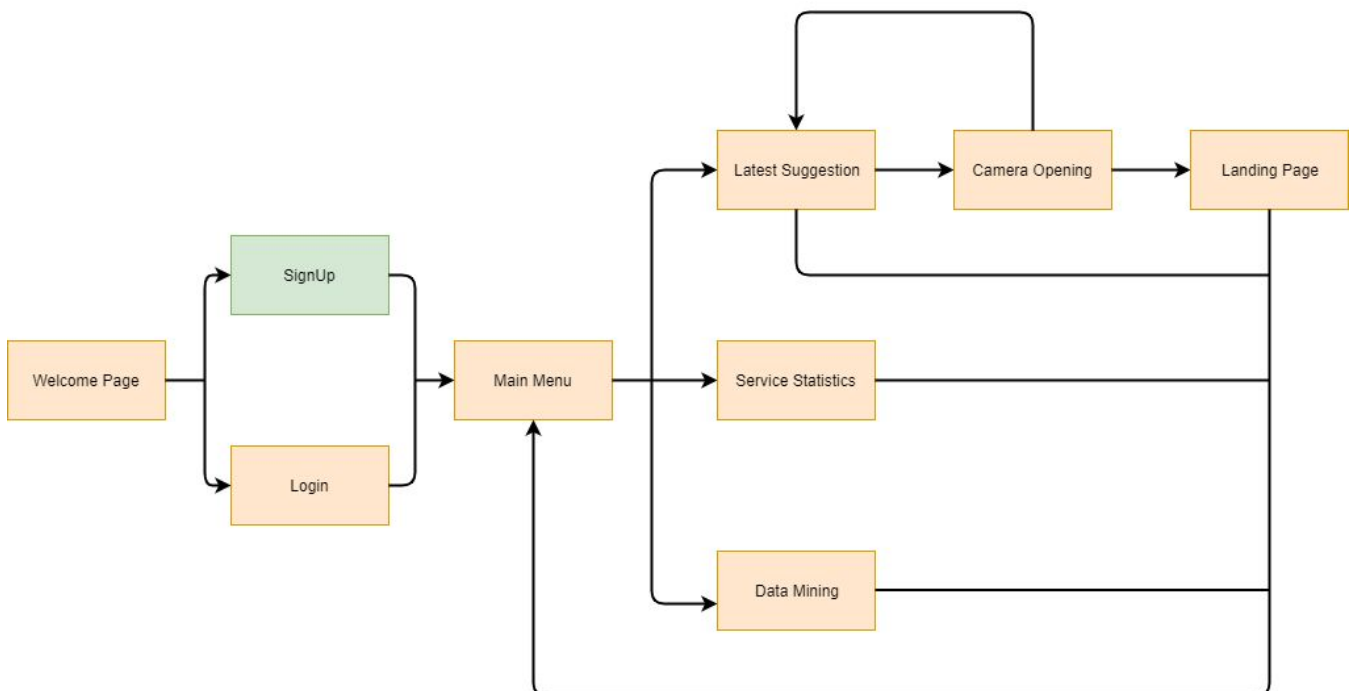


In order: report interface, personal reports, data mining interface for the user.

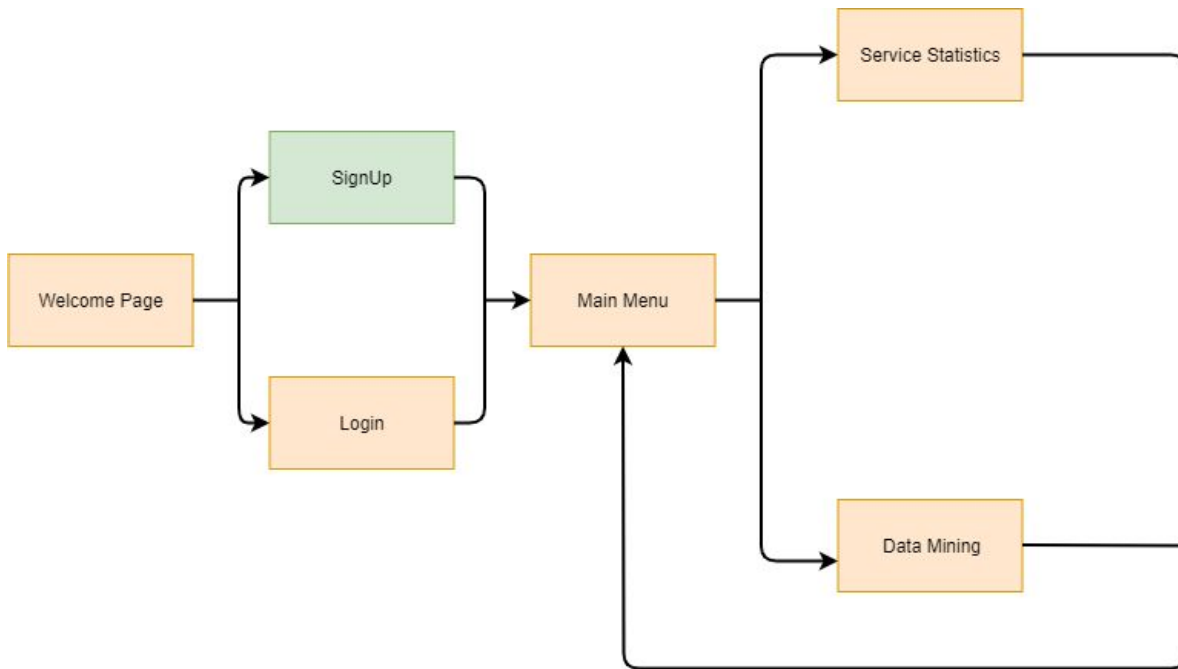


In order: data mining interface, suggestions tab and service statistics for municipalities.

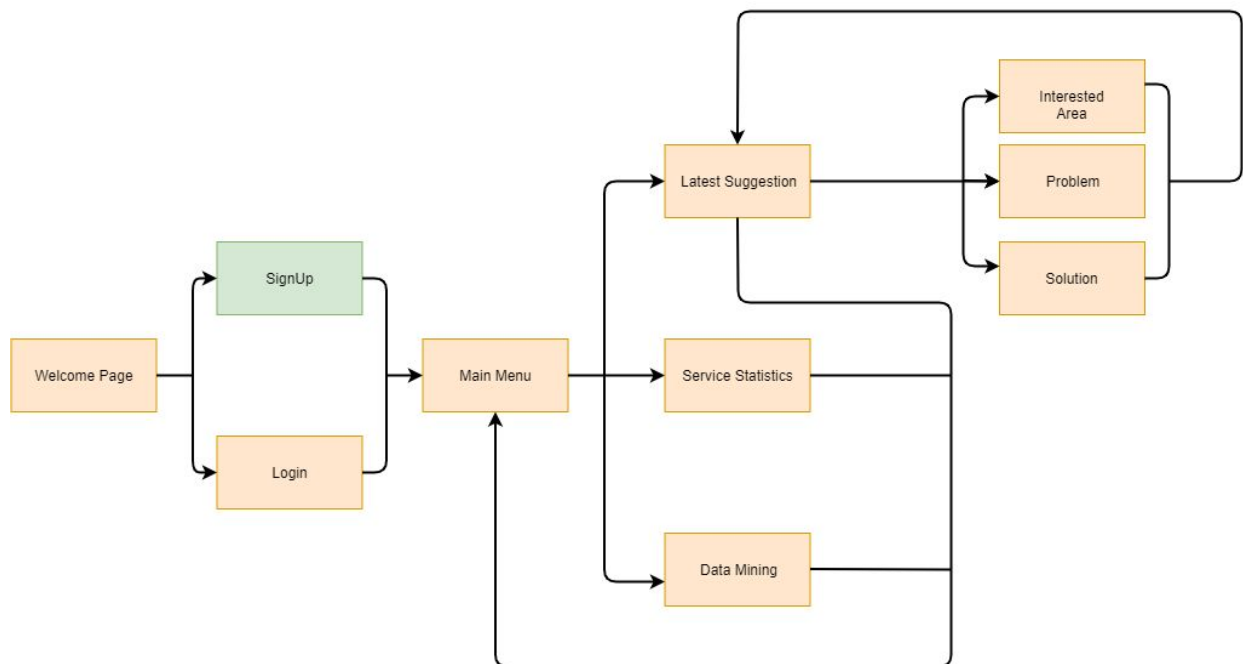
The following diagrams show the flow which the users will encounter using the application, according to the mocks previously presented:



UX diagram - normal user



UX diagram - authority



UX diagram - municipality

3.1.2 Hardware interfaces

The system has no hardware interfaces, it leans on customer's devices.

3.1.3 Software interfaces

Given the range of features the system offers, it will need to have some software interfaces.

First of all there is the need to have the possibility to store all reports and all the users' data. In this sense the system will need to use some external relational DBMS and their API to manage data.

The system will also need a service that will manage the GPS information and shares the position of the reports on the maps. This will be achieved using some API that can offer this service. It will probably be involved the Google Maps Geolocation API.

By the user's point of view the system will run on a simple app and all the interaction will be possible through it.

3.1.4 Communication interfaces

How the communication will be implemented is not important although the communication between different parts of the system is fundamental since it is tough to let user and municipalities communicate. However users and municipalities will make use of the usual communication protocols (TCP/IP) to guarantee the connection between their devices and the services' backend systems.

3.2 Functional requirements

3.2.1 User

Scenario 1

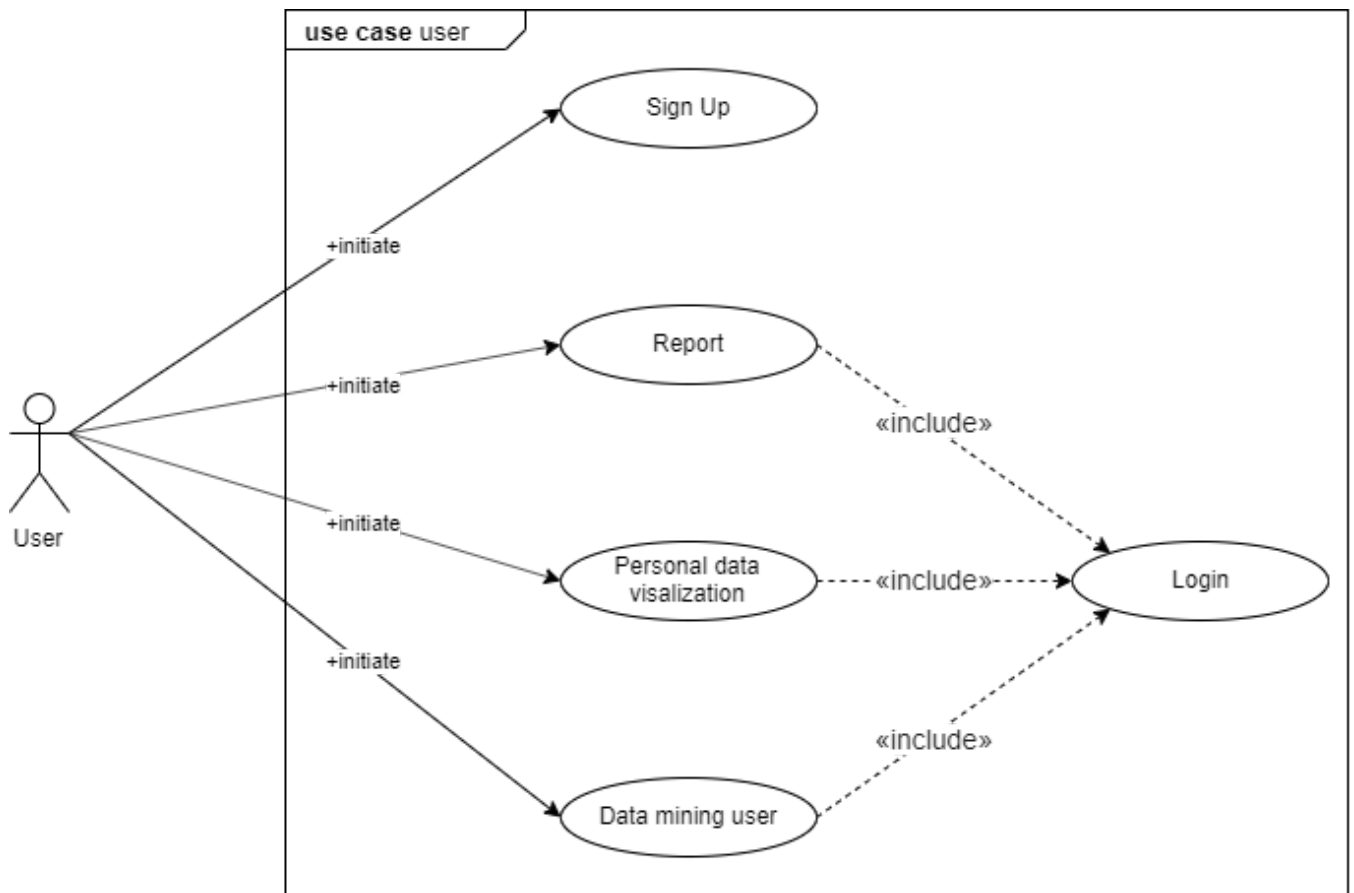
One day Alice is going to school by bike on the bike path as always. At some point the road is barred by a car parked exactly on the bike path. The girl decides to use her device using *SafeStreets*, an app installed on it. She takes a photo of the license plate, she adds all the necessary informations needed to report that violation and

then she sends the pic and the informations through the app. After that she is able to see in the application's archive the report she just sent.

Scenario 2

In August Mario arrives in Bari and wants to find an area where he won't struggle to park. He decides to use *safestreets* to get this information thinking that an higher rate of violations is associated to a lack of parkings. He looks for info about Bari and he discovers what are Bari's areas with the highest parking violations frequency and the most double-parked cars. Thanks to this Mario knows which zones he has to avoid. The application doesn't show all the details about vehicles and license plates connected to the violations.

Use Case Diagram



U.C.1

Name	Sign Up
Actor	User
Entry Condition	1. The user has downloaded and opened the application
Events flow	1. The user taps on the sign up option 2. The user fills all the fields (all mandatory) 3. The user confirms his data
Exit conditions	The user confirms his account through email link and system stores his data
Exceptions	1. The user hasn't filled all the fields 2. The user provided wrong data (invalid email) 3. The user inserts an already taken username or email 4. The user password doesn't meet the security requests

U.C.2

Name	Login
Actor	User
Entry Condition	1. The user has downloaded and opened the app. 2. The user has already done the "Sign Up" activity.
Events flow	1. The user chooses the "Login" option. 2. The user enters username and password in the respective fields.

	3. The user chooses the confirmation option.
Exit conditions	The user is logged in and he can execute other actions
Exceptions	1. The user enters the wrong username 2. The user enters the wrong password In both cases the user is warned by the application about what is the wrong field and receive the suggestion to update it and retry.

U.C.3

Name	Report
Actor	User
Entry Condition	1. The user has already done the "Login" activity.
Events flow	1. The user choose the "Report violation" option. 2. The user shots a pic of the violation 3. The system adds technical informations to the pic (e.g. data, time, position, ..) 4. The user adds the violation type 5. The user adds optional informations to the picture. 6. The report is created

Exit conditions	1. SafeStreets generates the report
Exceptions	1. GPS is not available. 2. Internet connection is not available 3. Invalid image inserted In both cases the user is notified about the issue.

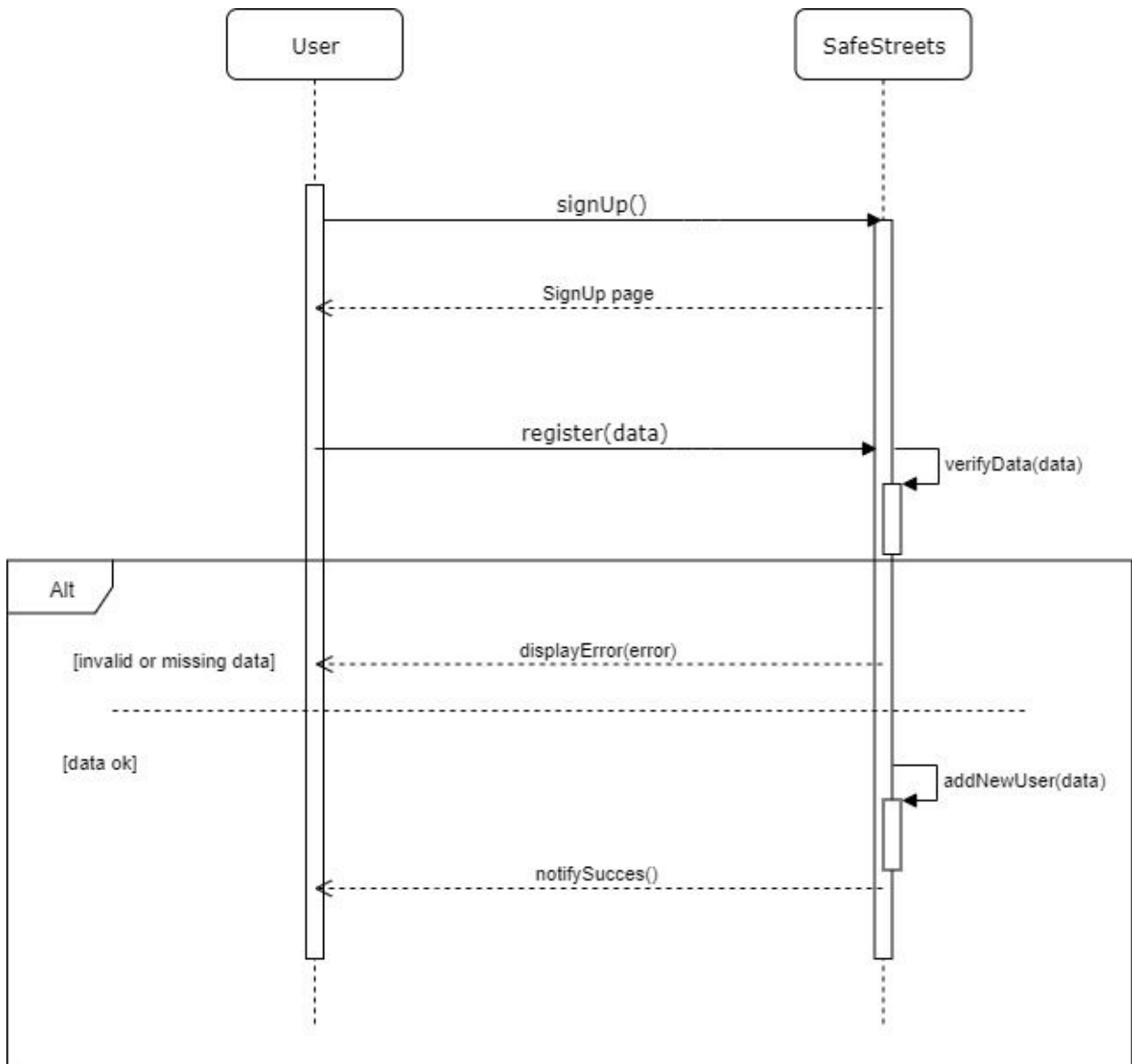
U.C.4

Name	Personal data visualization
Actor	User
Entry Condition	1. The user has already done the “Login” activity.
Events flow	1. The user taps on “My reports” 2. The user chooses among the available sent reports 3. The user visualizes his data
Exit conditions	The user taps on close button
Exceptions	1. No available reports

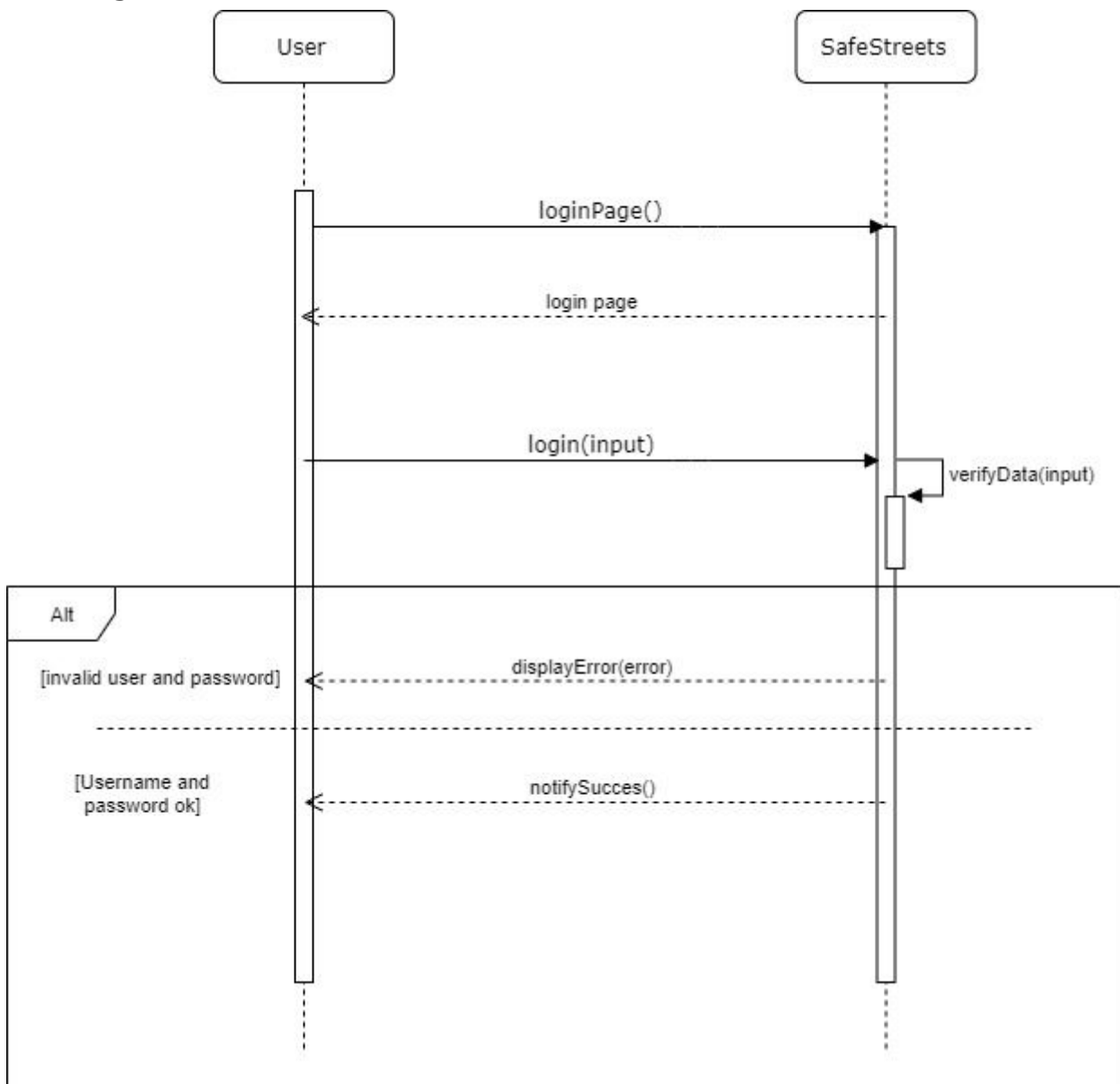
U.C.5

Name	Data mining
Actor	User
Entry Condition	1. The user has already done the “Login” activity.
Events flow	1. The user chooses the “Data mining” option. 2. The user select an area of interest. 3. The system collects all the report and builds the data to show 4. The information about that area is shown on the map through different highlightings.
Exit conditions	The user taps on close button
Exceptions	1. No available data 2. Invalid query

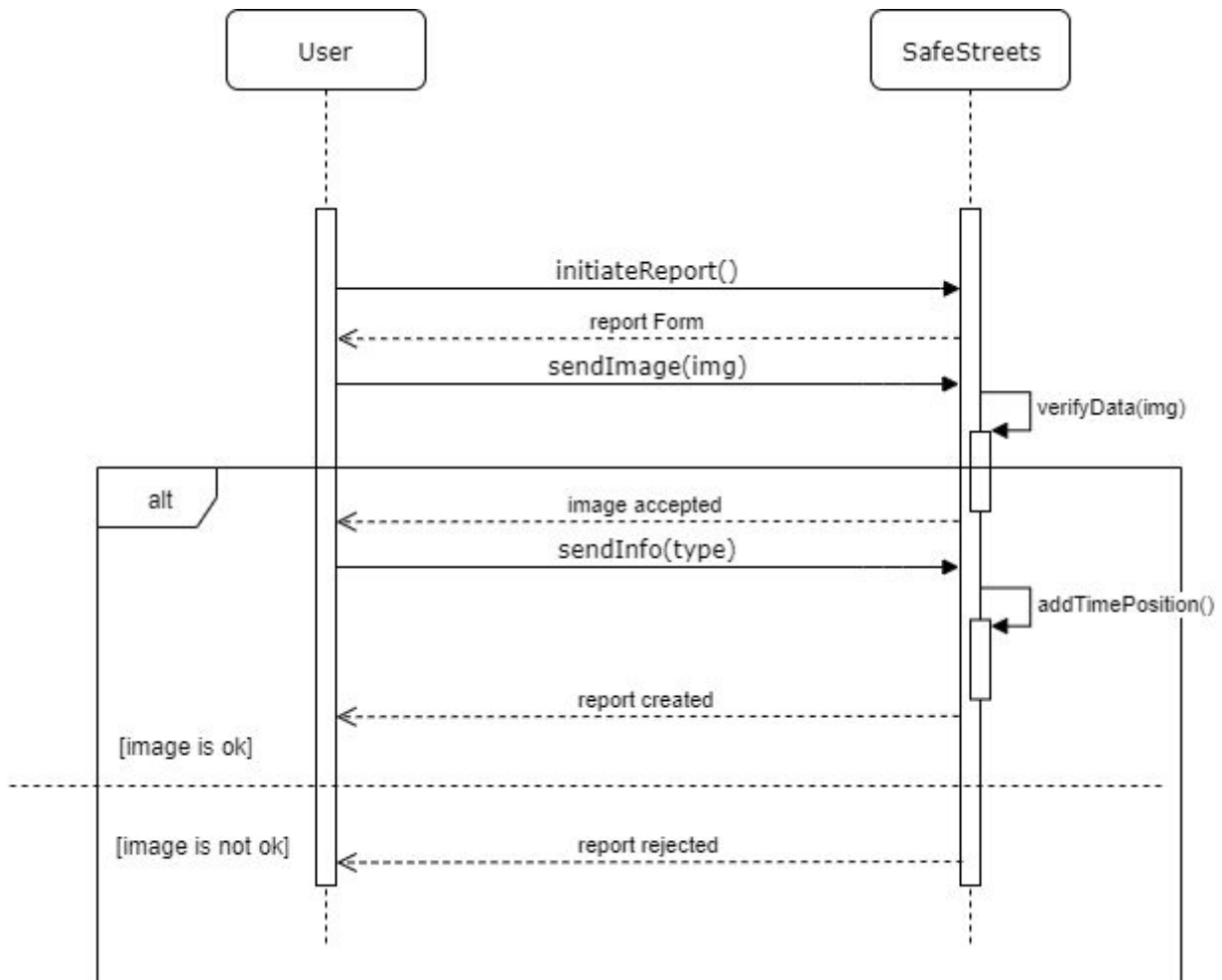
User SignUp



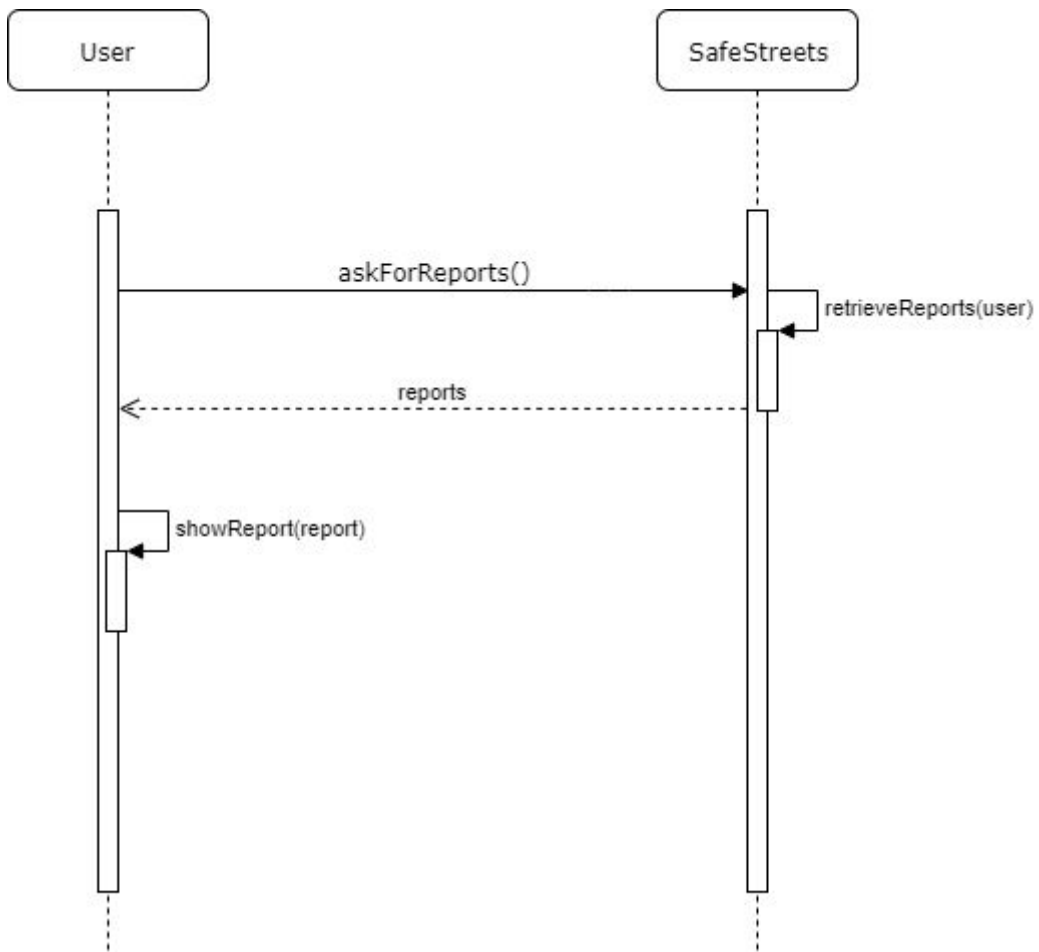
User Login



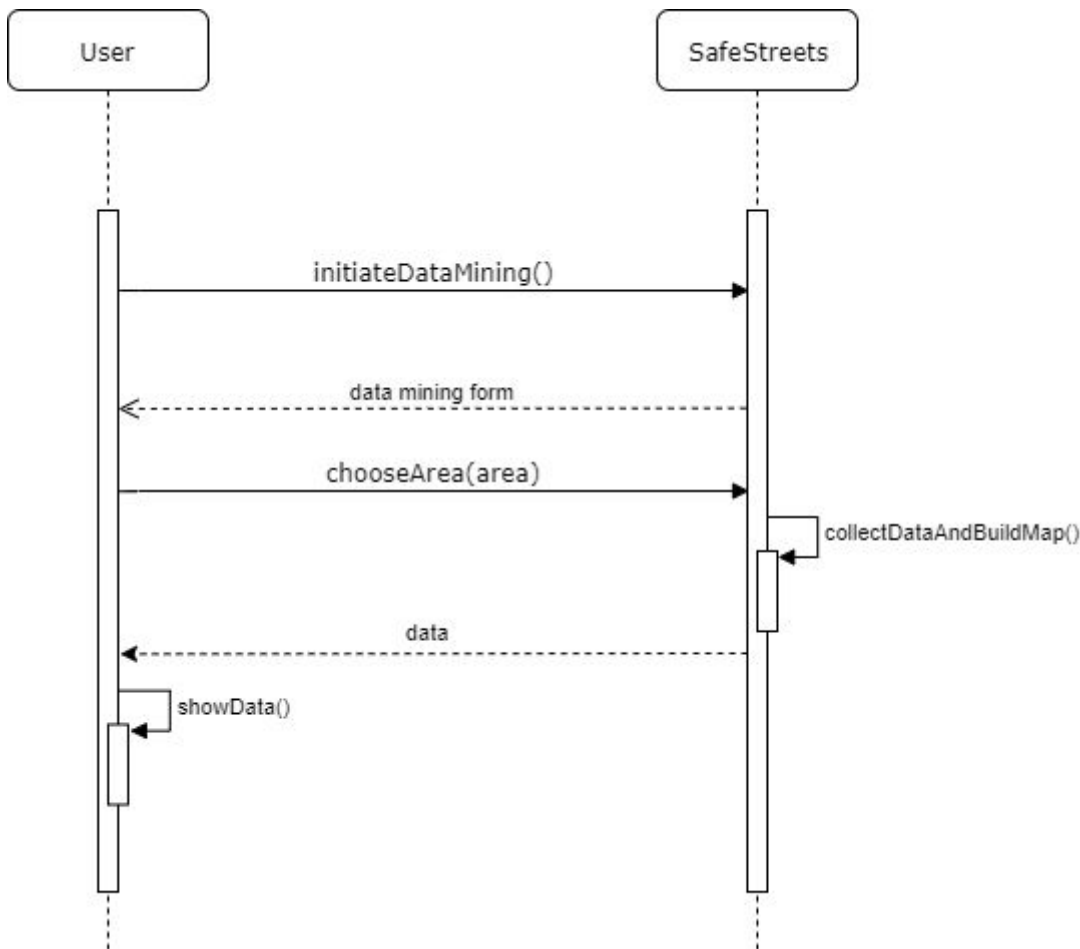
User Report Violation



User Personal Data Visualization



User Data mining



3.1.2 Authority

Scenarios

Scenario 3

A police unit has to patrol some areas in Terni and needs to understand where it is more likely to find parking violations. Thus, the unit decides to connect to *SafeStreets* using the police's account and looks for informations about Terni. The policemen using the service can see which are the Terni's areas connected to an higher probability of parking violations and they can also read not only what kind of vehicles committed violations often but also the license plates of the most recent infractions.

Scenario 4

Periodically *SafeStreets* ask the municipality to update his database and retrieves information about last accidents. The municipality of Milan periodically receives reports from *SafeStreets* indicating how many user-reported cars were involved in traffic accidents in the city centre. *SafeStreets* suggests a reduction of the speed limit in those areas. The Municipality, given the proximity of that area to places frequented by children (schools, parks), decides to execute the suggestion and to install some speed bumps.

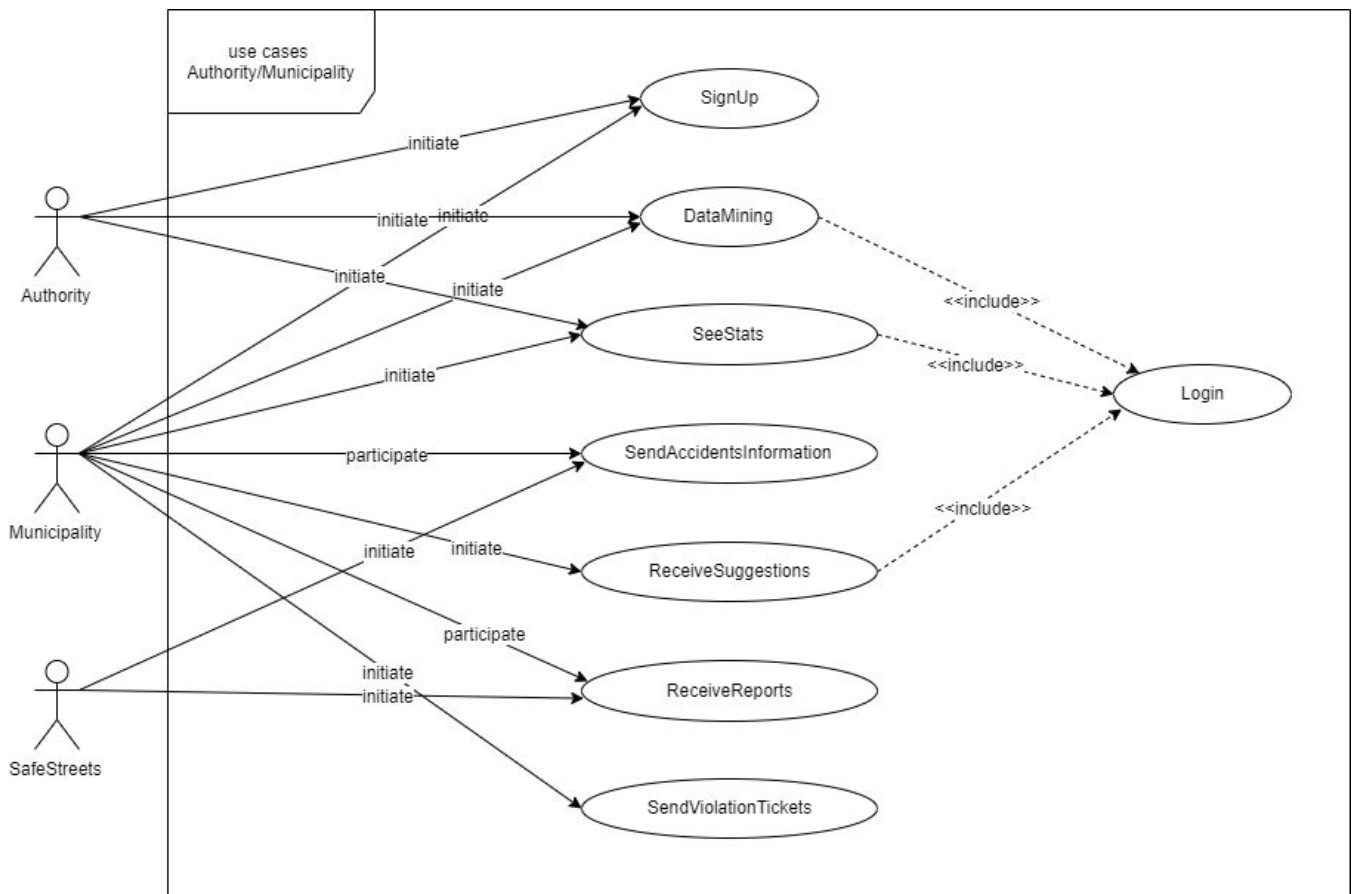
Scenario 5

The municipality of Genova decides to buy *SafeStreets* service in order to automate its penalty system for the city suburbs, barely supported by patrols. The municipality, once subscribed to the service, periodically receives the user made reports and forwards them to a special division which reviews and convalidates such reports, from home. Once the municipality has issued the violation tickets, it notifies *SafeStreets*. *SafeStreets* collects the information and updates its statistics about the efficiency of the service.

Scenario 6

Sandro is a mayoral candidate for the city of Messina and wants to propose in his election program the start of a collaboration between the city authorities (including the municipality) and *SafeStreets* to be re-elected. In order to convince many people during a press conference he decides to use the data provided by *SafeStreets* itself and show them in public. Therefore it connects to the service using the municipality account, collects the statistics and saves them on his computer.

Use Case Diagrams



U.C.6

Name	Sign Up
Actor	Authority
Entry Condition	The authority has downloaded and opened the application
Events flow	1. The authority taps on the sign up option 2. The authority fills all the fields (all mandatory) 3. The authority confirms his data
Exit conditions	The authority confirms his account through email link and system stores his data
Exceptions	1. The authority hasn't filled all the fields 2. The authority provided wrong data (fiscal code not matching other data, invalid email) 3. The authority inserts an already taken username 4. The authority password doesn't meet the security requests

U.C.7

Name	Login
Actor	Authority
Entry Condition	1. The authority has downloaded and opened the app. 2. The authority has already done the "Sign Up" activity.

Events flow	1. The authority chooses the “Login” option. 2. The authority enters username and password in the respective fields. 3. The authority chooses the confirmation option.
Exit conditions	The authority is logged in and he can execute other actions
Exceptions	1. The authority enters the wrong username 2. The authority enters the wrong password In both cases the authority is warned by the application about what is the wrong field and receive the suggestion to update it and retry.

U.C.8

Name	Data mining
Actor	Authority
Entry Condition	1. The authority has already done the “Login” activity.
Events flow	1. The authority chooses the “Data mining” option. 2. The authority select an area of interest. 3. The information about that area is shown on the map through different highlightings.
Exit conditions	The authority taps on close button
Exceptions	1. No available data

	2. Invalid query
--	------------------

U.C.9

Name	Send Accidents Information
Actor	Municipality
Entry Condition	1. The municipality has already done the "Sign up" activity. 2. The municipality has subscribed to the <i>SafeStreets</i> service. 3. A timer linked to such Municipality updates expires
Events flow	1. <i>SafeStreets</i> asks Municipality for accidents updates 2. Municipality sends updates about recent accidents 3. <i>SafeStreets</i> receives data and integrates with its own data 4. <i>SafeStreets</i> builds new suggestions with the new data
Exit conditions	<i>SafeStreets</i> correctly builds new suggestions for the Municipality

U.C.10

Name	Receive suggestions
Actor	Municipality
Entry Condition	1. The municipality has already done the "Sign up" activity. 2. The municipality has subscribed to the <i>SafeStreets</i> service.

Events flow	1. <i>Municipality</i> asks SafeStreets for suggestions 2. The Municipality is notified with the suggestions
Exit conditions	The Municipality receives the suggestions
Exceptions	1. Not enough data to perform analysis

U.C.11

Name	Receive reports
Actor	Municipality
Entry Condition	1. A registered Municipality uses the service for reporting a violation 2. A timer linked to such Municipality updates expires
Events flow	1. SafeStreets delivers the reports to the Municipality. 2. The Municipality can see and analyze the report.
Exit conditions	The authority taps on close button.
Exceptions	1. The report is forged.

U.C.12

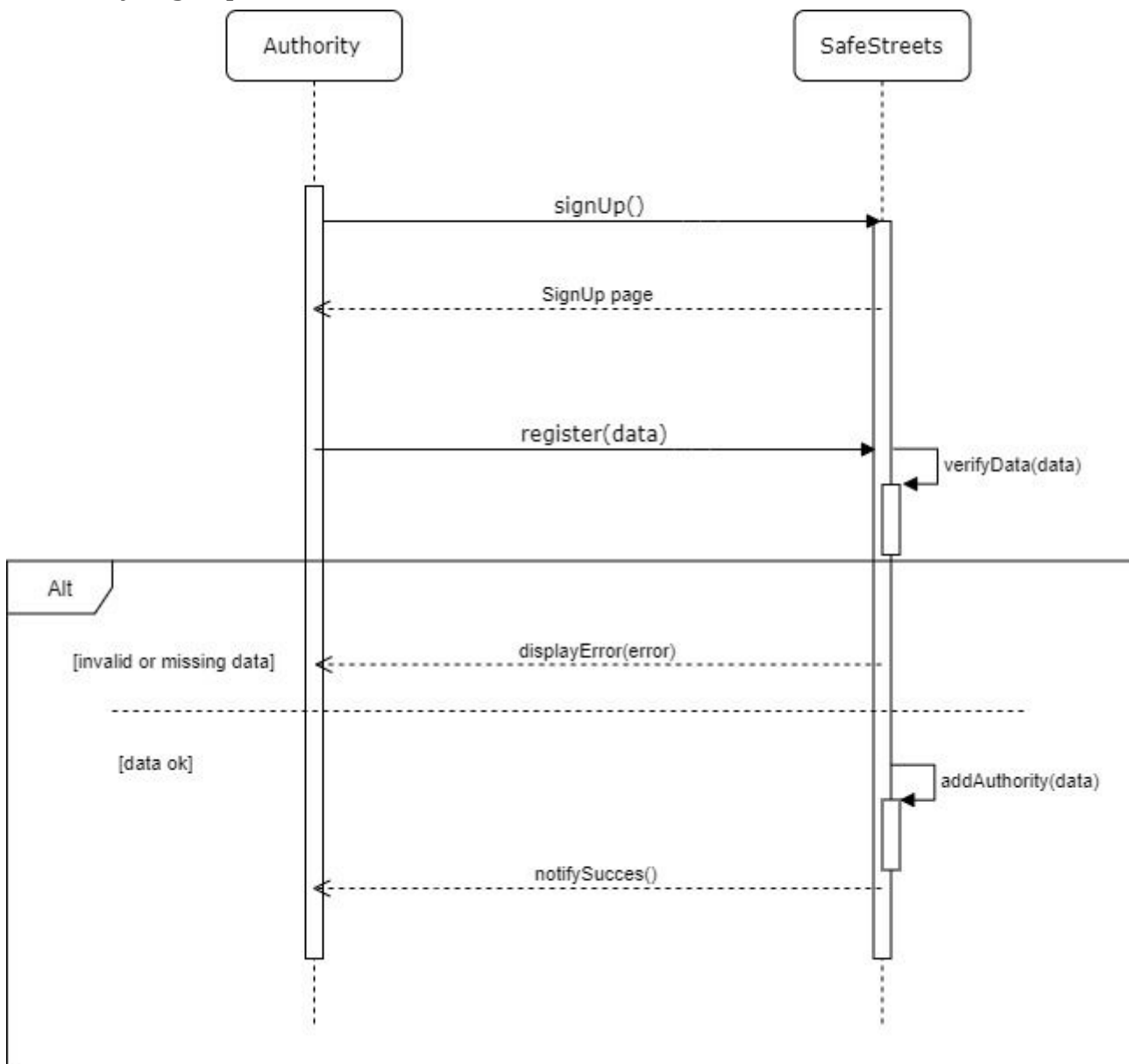
Name	Send Violation Tickets
Actor	Municipality
Entry Condition	1. A registered Municipality uses the

	service for reporting a violation 2. A timer linked to such Municipality updates expires
Events flow	1. The violation tickets are requested from SafeStreets through the dedicated API 2. The Municipality shares its violation tickets
Exit conditions	SafeStreets collects the information
Exceptions	1. No available data

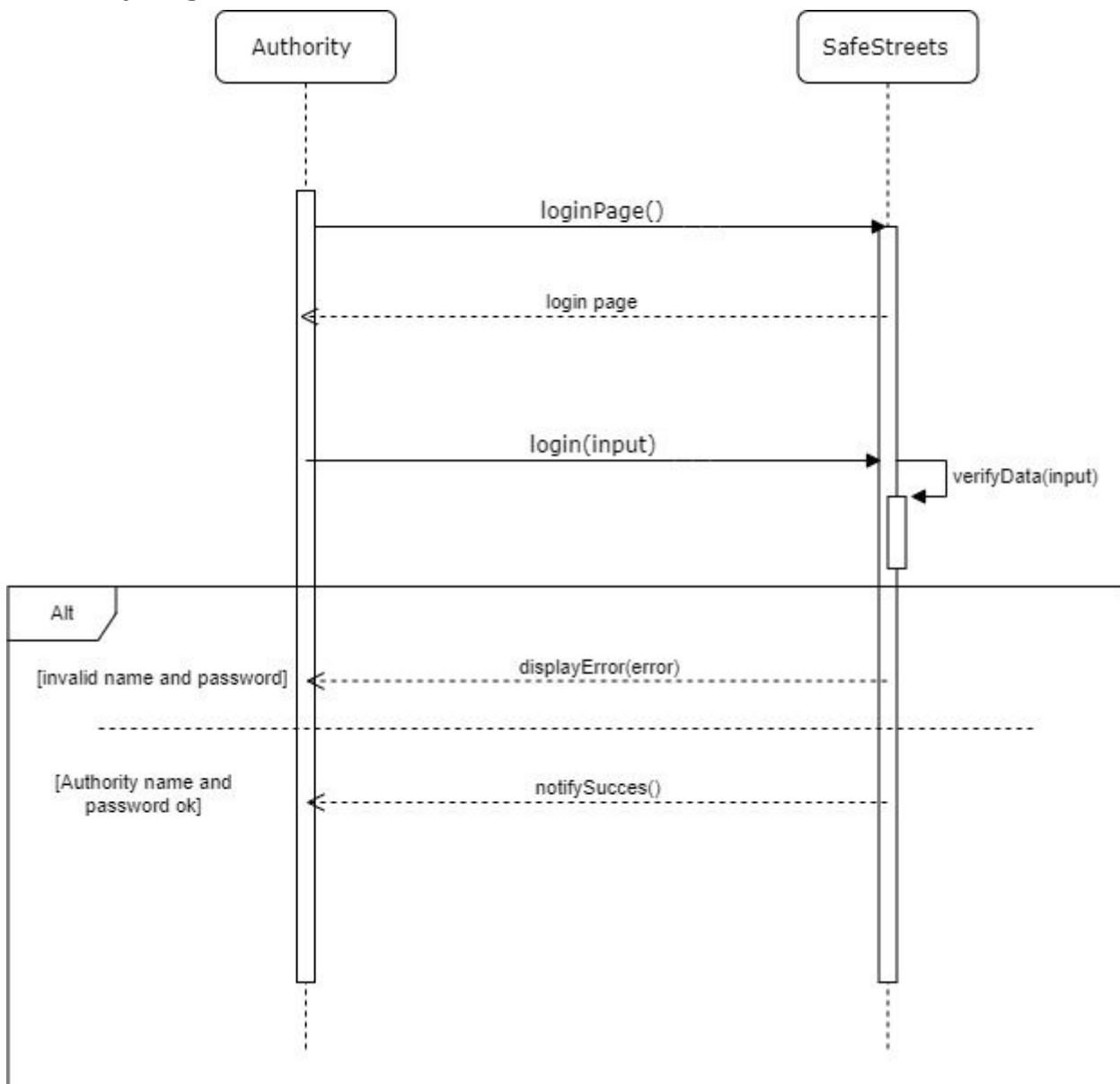
U.C.13

Name	See Stats
Actor	Authority
Entry Condition	3. The authority has already done the "Sign up" activity 4. The authority has already done the "login" activity
Events flow	3. The authority press the stats option. 4. SafeStreets builds the stats and updates them. 5. SafeStreets sends the stats. 6. The application show the statistics
Exit conditions	The authority saves the document and closes the app.

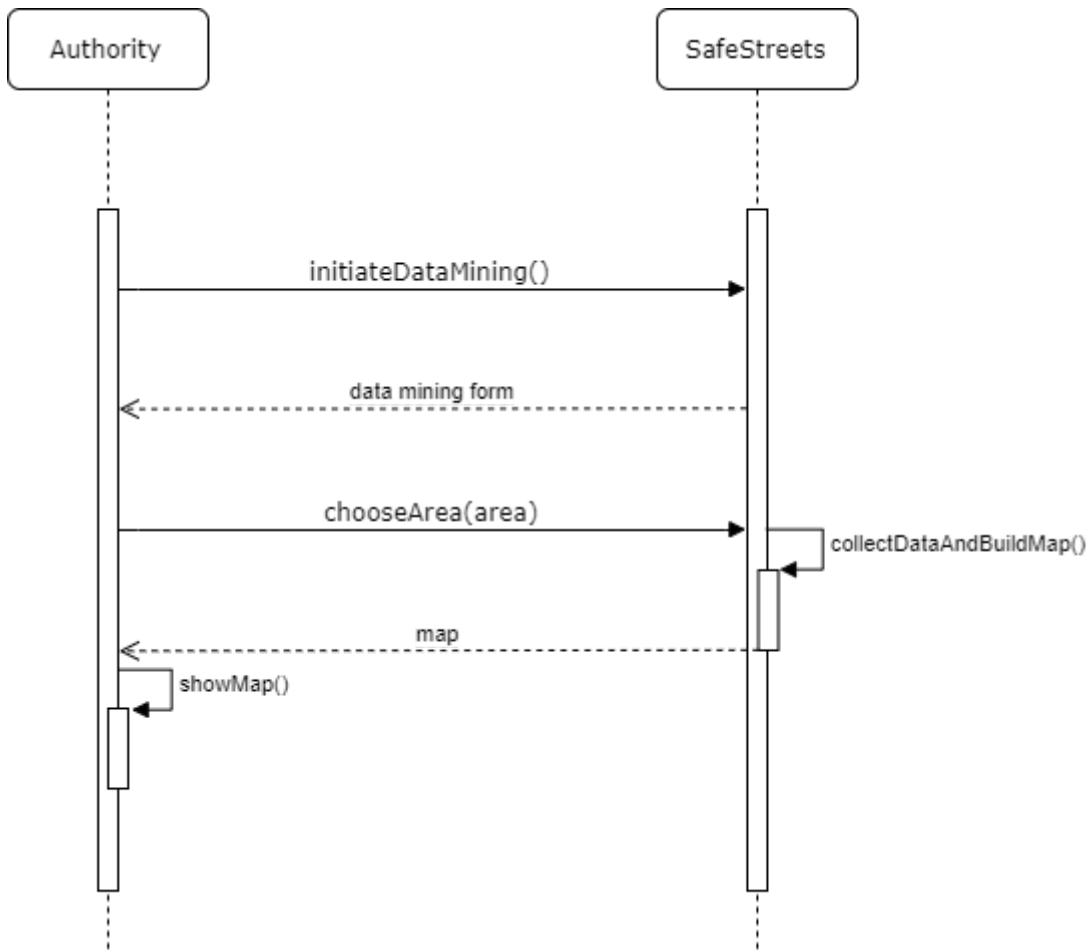
Authority SignUp



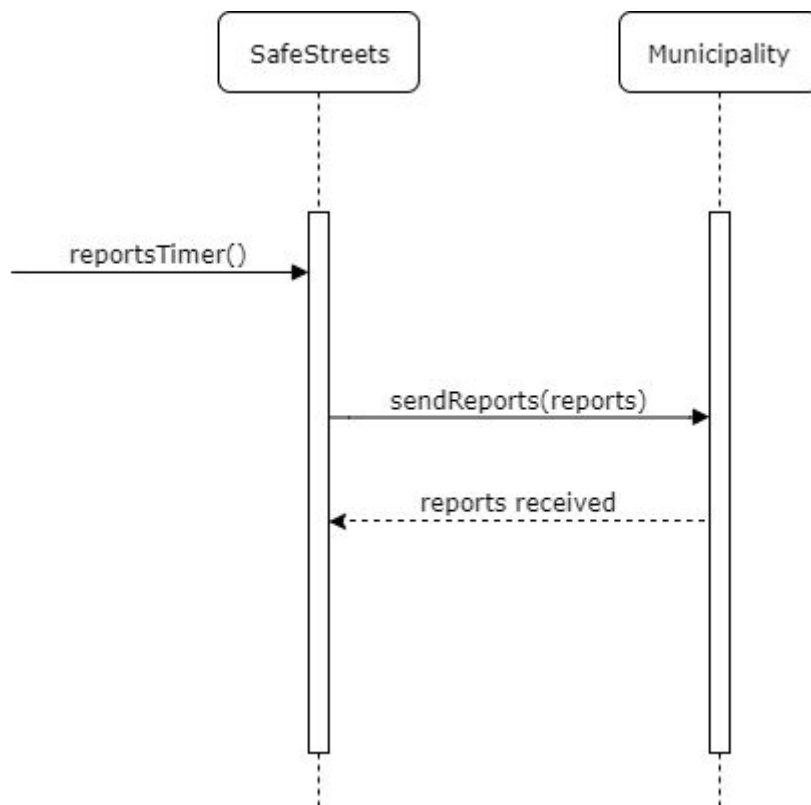
Authority Login



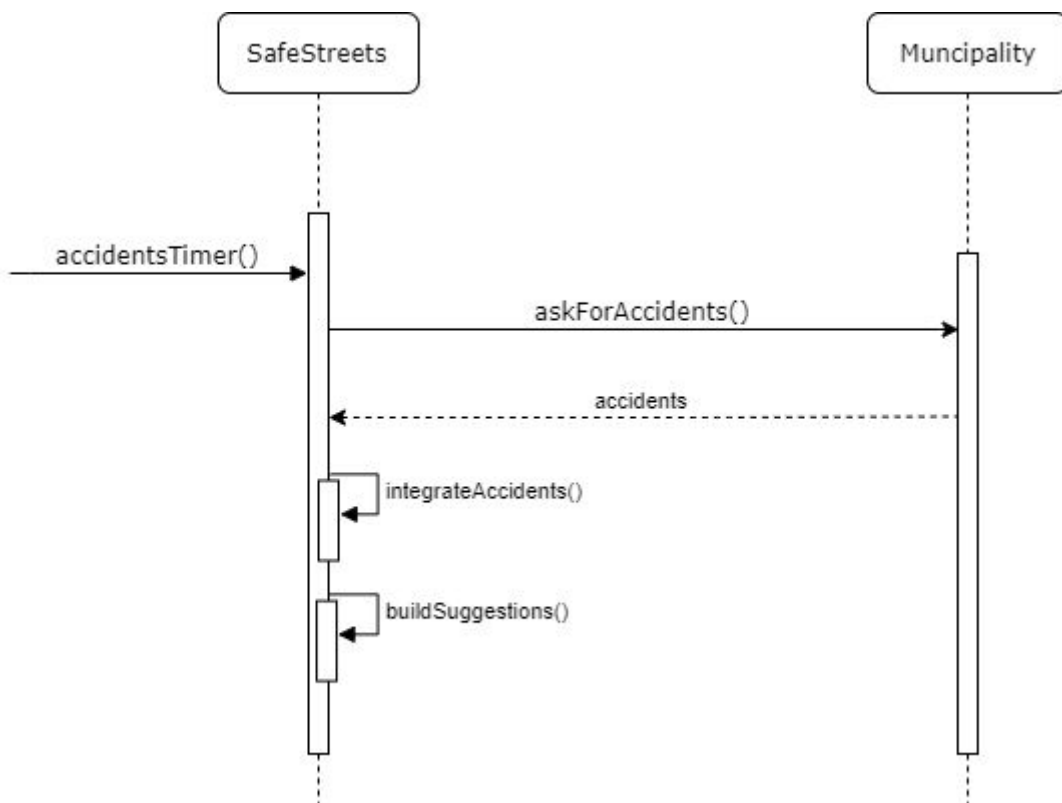
Authority performs data mining



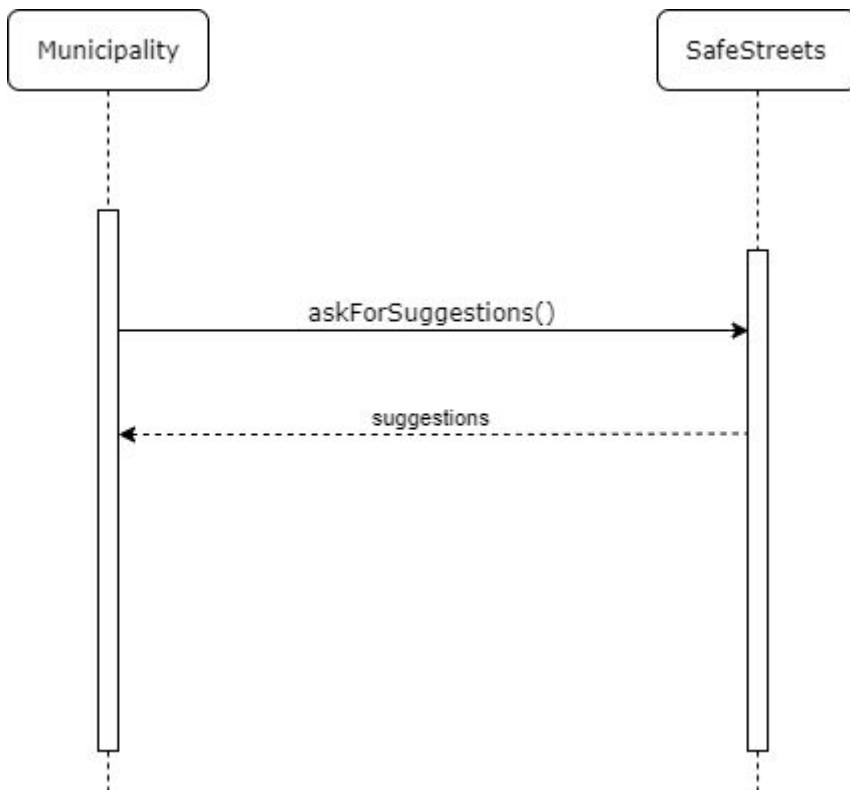
Municipality receives reports



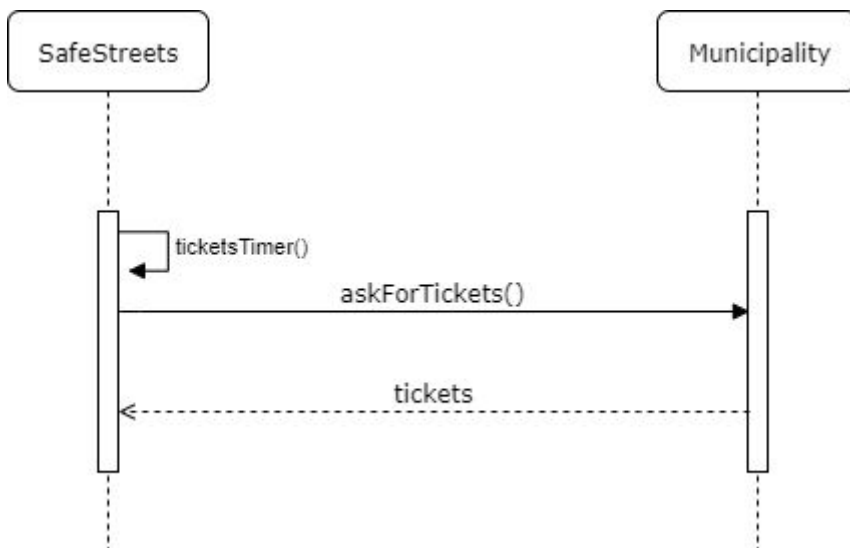
Send Accidents Information



Municipality receives suggestions



Municipality sends violation tickets



Authority sees stats



3.2.3 Requirements

In this section we aim to show that the requirements ensure the satisfaction of the goals in the context of the domain assumption. For this reason we divided goals, requirements and domain assumption as follow.

G1. The application must allow the user to send photos about parking violations

- **R01.** Safestreets takes the license plate from the photo sent by user.
- **R02.** The system must allow the user to insert the type of violation.
- **R03.** Safestreet stores the reports provided by users if the photo is not broken.
- **R04.** The system must use a gps system on the device to obtain the position of the violation.
- **D01.** Each license plate is a unique identifier.

- **D02.** The devices that acquire violations' position provides location with an error of 5 meters at most.
- **D03.** The internet connection works properly without failure.
- **D07.** The information coming from the users is never broken if the image is not manipulated and the license plate is readable.
- **D09.** The data and the time is set correctly on the device of the user and authorities.
- **D10.** Users can correctly identify the type of violation.

G2. The application must allow the user to consult his own data

- **R05.** The system must save the collected data of the users registered to SafeStreets in real time.
- **R06.** The system must allow the users and the authorities to access their own registered data and all the reports done with the possibility to consult them.
- **D03.** The internet connection works properly without failure.

G3. The application must allow the user and authorities to extract the data about about parking violations

- **R07.** The system must allow at the applicant to select the interested areas where the parking violations occurred.
- **R08.** The system must show the extracted data to the applicant.
- **D03.** The internet connection works properly without failure.

G4. The application must provide different levels of visibility of the requested data based on the role of the issuer

- **R09.** If the applicant is a user the system must show only the areas with the highest parking violations frequency.
- **R10.** If the applicant is an authority the system must show the areas with the highest parking violations frequency but also what kind of vehicles committed violations often and the license plates of the most recent infractions.
- **D05.** The authorities and in particular the municipality have demonstrated that they are acknowledged public institutions.

G5. The application must be capable of retrieving information from municipality (those offering the services) about accidents

- **R11.** The system must extract the data about the accidents of a territory from the municipality.
- **R12.** The system must extract the data about the accidents periodically.
- **D03.** The internet connection works properly without failure.
- **D05.** The authorities and in particular the municipality have demonstrated that they are acknowledged public institutions.
- **D04.** The information about the accidents of the municipality are correct.
- **D06.** Municipality is a particular case of authority.
- **D08.** Each municipality that shares information with SafeStreets has authorized the app to take the informations that has inside.

G6. The application must provide the municipality suggestions about possible interventions to violations

- **R13.** The system must send the suggestions to the municipality.
- **R14.** The system must create suggestions for the municipality depending on the information received.
- **R15.** The system must create the suggestions periodically.

- **D03.** The internet connection works properly without failure.
- **D05.** The authorities and in particular the municipality have demonstrated that they are acknowledged public institutions.

G7. The application must allow the municipality to retrieve the information about the reports to generate traffic tickets.

- **R16.** The system must create and make available the information about the violation on a territory for the municipality.
- **D03.** The internet connection works properly without failure.
- **D05.** The authorities and in particular the municipality have demonstrated that they are acknowledged public institutions.
- **D06.** Municipality is a particular case of authority.

G8. The application must be capable of retrieving information from municipality about issued tickets.

- **R17.** The system must extract the data about the issued tickets periodically from municipality.
- **R18.** The system will use the data about the issued tickets to generate stats about the efficiency of the service.
- **D03.** The internet connection works properly without failure.
- **D05.** The authorities and in particular the municipality have demonstrated that they are acknowledged public institutions.
- **D08.** Each municipality that shares information with SafeStreets has authorized the app to take the informations that has inside.

G9. The application must allow the authority to see stats about the efficiency of the service.

- **R19.** The system must build stats to show at the authority the trends in the issuing of tickets, the most egregious offenders and some parameters about the efficiency of SafeStreets to report the violations.
- **D03.** The internet connection works properly without failure.
- **D05.** The authorities and in particular the municipality have demonstrated that they are acknowledged public institutions.

3.2.4 Traceability matrix

Goal	Requirement	Use Cases
G1	R1	U.C.3
G1	R2	U.C.3
G1	R3	U.C.3
G1	R4	U.C.3
G2	R5	U.C.4
G2	R6	U.C.4
G3	R7	U.C.5, U.C.8
G3	R8	U.C.5, U.C.8
G4	R9	U.C.5
G4	R10	U.C.8
G5	R11	U.C.9
G5	R12	U.C.9
G6	R13	U.C.10
G6	R14	U.C.10
G6	R15	U.C.10
G7	R16	U.C.11
G8	R17	U.C.12

G8	R18	U.C.12
G9	R19	U.C.13

3.3 Performance requirements

The system must allow its use to a great number of users and many municipalities. It must allow quick response in delivery time for the reports. Municipality could give access to a huge database that the systems needs to manage. Data losses need to be minimized if not made absent. Information integrity must be granted.

3.4 Design constraints

3.4.1 Standard compliance

The system, in order to protect the privacy of its users, complies with the new GDPR (General Data Protection Regulation). The data stored in the app will be protected and shared only with the permission of the individual that owns this data. Only municipalities can access detailed informations about reports in their pertinence area.

3.4.2 Hardware limitations

The system requires the user to have a smartphone or a pc in order to use the service. Users must have a smartphone with Android or IOS system, they must have a GPS and an internet connection and a camera on it in order to create reports. Authorities only need an internet connection.

3.4.3 Any other constraint

The application will ask for user's position in order to receive trustable reports.

3.5 Software system attributes

3.5.1 Reliability

The system should be available 24/7. It should run continuously without interruptions. Thus, it has to be fault tolerant. The DB should implement some RAID architecture in order to guarantee reliability.

3.5.2 Availability

As mentioned before, availability can be obtained through fault tolerance by data duplication. In particular, some services must have an higher grade of availability guaranteed. User are expected to be able to report violations 99.9% of the time while information mining, receiving suggestions and automatic ticketing can afford a 95% of up-time.

3.5.3 Security

Given that the internal DB contains sensitive information about SafeStreets' users and not, the system should be protected from any external attack in order to guarantee privacy and consistency of data. In this sense SafeStreets should implement cryptographic systems to ensure the aforementioned qualities. The data transmission should also be protected in a similar way.

3.5.4 Maintainability

The service will be flexible and easy to be maintained. It must facilitate additions of new features or options. It will be used clear code, design patterns will be followed and a detailed documentation will be provided. The system will be tested and the tests should have a 90% of code coverage

3.5.5 Portability

The application and the website should be responsive, so that an user can access the service using both smartphones and tablets. The application should work both on Android and on IOS.

4. Formal analysis using Alloy

```
abstract sig DataMining{}

sig Street{}

sig Username{}

sig Password{}

sig LicensePlate{}

sig TypeOfViolation{}

sig VehicleType{}

sig Date{}

sig TimeStamp{}

sig Name{}

sig Surname{}

sig City{}

sig TypeOfAuth{}

sig TypeOfSuggestion{}

sig NameAuth{}

sig Code{}

sig UnsafeArea
{
  nameOfStreet : one Street
}

sig Registration
{
  username: one Username,
  password: one Password
}

sig InformationForUser extends DataMining
{
  frequencyViolation:one Int,
```

```

    area : one UnsafeArea
  }
  {
    frequencyViolation >= 0 -- Frequency must be >= 0
  }

sig InformationForAuth extends DataMining
{
  frequencyViolation:one Int,
  lastViolations: set ParkingViolation,
  area : one UnsafeArea
}
{
  frequencyViolation >= 0 -- Frequency must be >= 0
}

abstract sig GeneralUser
{
  credentials: one Registration
}

sig User extends GeneralUser
{
  name: one Name,
  surname: one Surname,
  birthday: one Date,
  informationMined: set InformationForUser
}

sig PictureOfViolation
{
  date: one Date,
  time:one TimeStamp,
  isManipulated: one Int --it is being used like a boolean to indicate if the image is
manipulated or not
}
{
  isManipulated = 0 or isManipulated = 1 --is Manipulated means True or False
}

sig ParkingViolation
{
  licensePlate: one LicensePlate,
  typeOfViolation: one TypeOfViolation,
  nameOfStreet: one Street,
  vehicleType: one VehicleType,
  picture: one PictureOfViolation,
  user: one User --user that notified the parking violation
}

sig Authority extends GeneralUser

```

```

{
  typeOfAuth: one TypeOfAuth,
  nameAuth: one NameAuth,
  code: one Code,
  informationMined: set InformationForAuth
}

sig Municipality extends Authority
{
  city: one City,
  accidents: set Accident,
  suggestions: set Suggestion,
  tickets: set TrafficTicket
}

sig Accident
{
  nameOfStreet: one Street
}

sig Suggestion
{
  typeOfSuggestion: one TypeOfSuggestion,
  interventions: some PossibleInterventions
}

sig PossibleInterventions
{
}

sig TrafficTicket
{
  municipality: one Municipality,
  violation: one ParkingViolation
}

sig Statistic
{
  efficiency: one Int,
  nIssuedTickets: one Int
}
{
  efficiency >= 0 and efficiency <= 100 and nIssuedTickets >= 0
}

fact NoDuplicatedUsernames
{
  no disj g1, g2: GeneralUser | g1.credentials.username = g2.credentials.username
}

fact AllUsernamesBelongToARegistration

```

```

{
  all u: Username | some r: Registration | u in r.username
}

fact AllRegistrationAreAssociatedWithAUser
{
  all r: Registration | some g: GeneralUser | r in g.credentials
}

fact AllStreetsBelongToASomething
{
  all s: Street | some u: UnsafeArea, p: ParkingViolation, a: Accident | s in
u.nameOfStreet or s in p.nameOfStreet or s in a.nameOfStreet
}

fact NoDuplicatedUnsafeAreas
{
  no disj u1, u2: UnsafeArea | u1.nameOfStreet = u2.nameOfStreet
}

fact AllPasswordArePartOfACredential
{
  all p: Password | some r: Registration | p in r.password
}

fact AllNameAreInUser
{
  all n: Name | some u: User | n in u.name
}

fact AllSurnameAreInUser
{
  all s: Surname | some u: User | s in u.surname
}

fact AllBirthdateAreInUser
{
  all d: Date | some u: User, p: PictureOfViolation | d in u.birthday or d in p.date
}

fact AllTimestampArePartOfAPictureOfViolation
{
  all t: Timestamp | some p: PictureOfViolation | t in p.time
}

fact AllLicensePlateBelongParkingViolation
{
  all l: LicensePlate | some p: ParkingViolation | l in p.licensePlate
}

fact AllVehicleTypeBelongParkingViolation

```

```

{
  all v: VehicleType | some p: ParkingViolation | v in p.vehicleType
}

fact AllTypeOfViolationsBelongParkingViolation
{
  all t: TypeOfViolation | some p: ParkingViolation | t in p.typeOfViolation
}

fact AllPictureOfViolationsBelongParkingViolation
{
  all p1: PictureOfViolation | some p2: ParkingViolation | p1 in p2.picture
}

fact NoDuplicatedParkingViolations
{
  no disj p1, p2: ParkingViolation | p1.picture = p2.picture
}

fact AllTypesOfAuthorityBelongToAnAuthority
{
  all t: TypeOfAuth | some a: Authority | t in a.typeOfAuth
}

fact AllNamesOfAuthorityBelongToAnAuthority
{
  all n: NameAuth | some a: Authority | n in a.nameAuth
}

fact AllAuthorityCodesAreUnique
{
  no disj a1, a2: Authority | a1.code = a2.code
}

fact AllCodesBelongToAnAuthority
{
  all c: Code | some a: Authority | c in a.code
}

fact AllCitiesBelongToAMunicipality
{
  all c: City | some m: Municipality | c in m.city
}

fact noDuplicatedMunicipalities
{
  no disj m1, m2: Municipality | m1.city = m2.city
}

fact AllSuggestionsBelongToAMunicipality
{

```

```

all s: Suggestion | some m: Municipality | s in m.suggestions
}

fact AllAccidentsBelongToAMunicipality
{
  all a: Accident | some m: Municipality | a in m.accidents
}

fact AllTypesOfSuggestionBelongToASuggestion
{
  all t: TypeOfSuggestion | some s: Suggestion | t in s.typeOfSuggestion
}

fact AllPossibleInterventionsBelongToASuggestion
{
  all p: PossibleInterventions | some s: Suggestion | p in s.interventions
}

fact LastViolationImpliesExistenceOfFrequencyViolation
{
  all info: InformationForAuth | #info.lastViolations <= info.frequencyViolation
}

fact AllElementsInLastViolationsMustBeInInformationForAuthArea
{
  all info: InformationForAuth | all violation: info.lastViolations |
violation.nameOfStreet = info.area.nameOfStreet
}

fact AllParkingViolationsHaveNotManipulatedPictures
{
  all violation: ParkingViolation | violation.picture.isManipulated = 0
}

fact DuplicatedLicensePlateMeansSameVehicleType
{
  all v1: ParkingViolation, v2: ParkingViolation | v1.licensePlate = v2.licensePlate
implies v1.vehicleType = v2.vehicleType
}

fact AllTicketsInAMunicipalityHasCorrectMunicipality
{
  all m: Municipality | all t: m.tickets | m = t.municipality
}

fact ConsistencyInEfficiencyAndNumberOfTickets
{
  all s: Statistic | s.efficiency != 0 implies s.nIssuedTickets != 0 and s.nIssuedTickets
!= 0 implies s.efficiency != 0
}

```

```

}

fact StatisticsSingleton1
{
  all s: Statistic | s.nIssuedTickets = #TrafficTicket
}

fact CheckOnCardinalityOfFrequencyViolations
{
  all ia: InformationForAuth | ia.frequencyViolation = #ia.lastViolations
}

```

Instance 1

```

pred world1{
#TrafficTicket = 1
#User = 1
#Authority = 1
#ParkingViolation = 1
#Street = 1
}

run world1 for 2

```

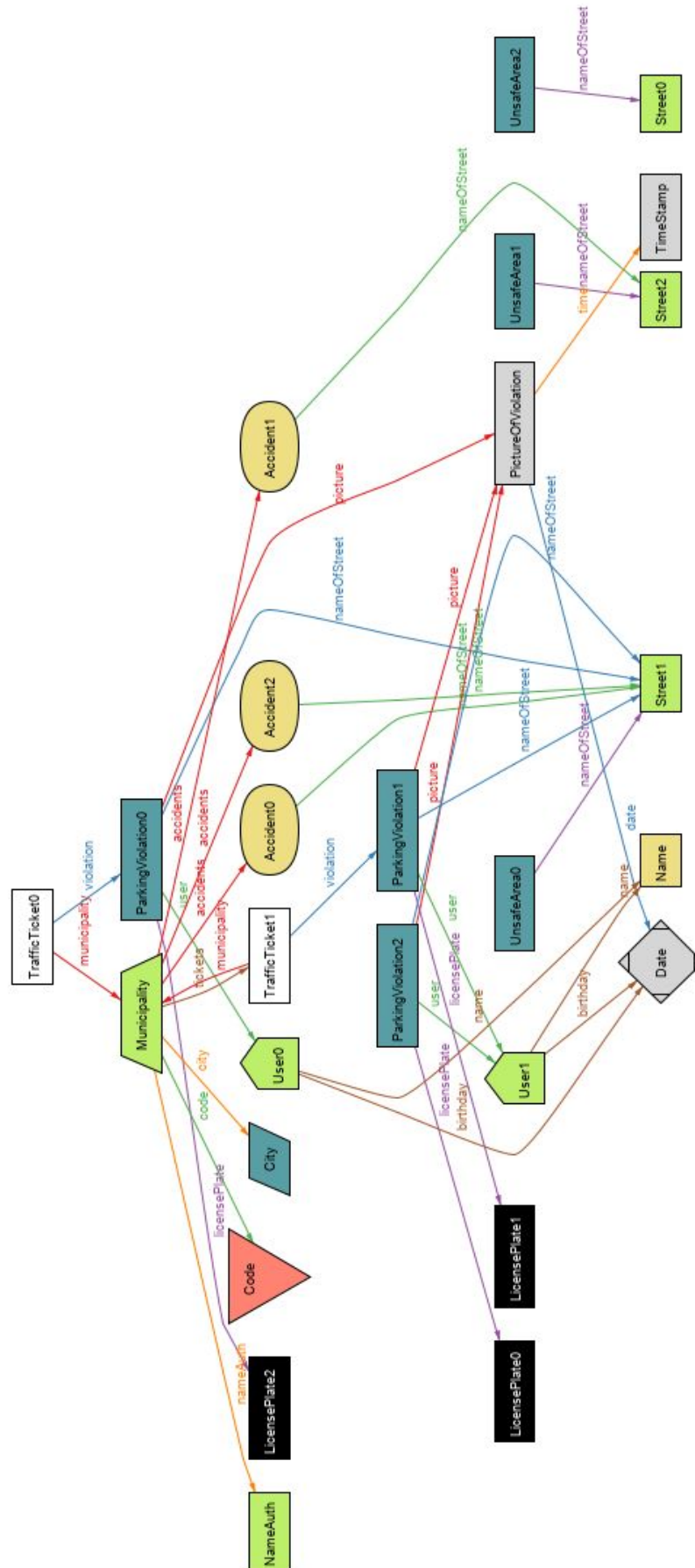
Executing "Run world1 for 2"

```

Solver=sat4j Bitwidth=4 MaxSeq=2 SkolemDepth=1 Symmetry=20
4729 vars. 438 primary vars. 7878 clauses. 41ms.
Instance found. Predicate is consistent. 47ms.

```

This is a generic instance containing few entities of each signature. We included a `TrafficTicket`, a `User`, an `Authority`, a `ParkingViolation` and a `Street` so that the instance is complete but not difficult to understand.



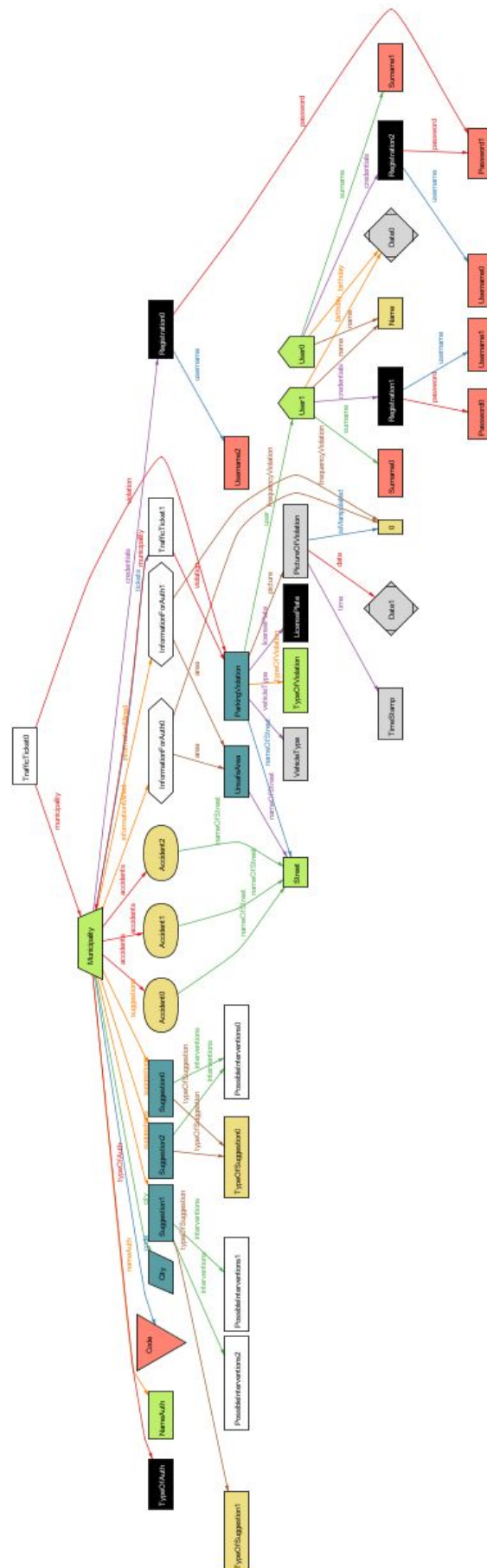
Instance 2

```
pred world2{  
  #TrafficTicket = 2  
  #User = 2  
  #Authority = 1  
  #ParkingViolation = 1  
  #Street = 1  
}  
  
run world2 for 3
```

Executing "Run world2 for 3"

```
Solver=sat4j Bitwidth=4 MaxSeq=3 SkolemDepth=1 Symmetry=20  
9483 vars. 822 primary vars. 15514 clauses. 38ms.  
Instance found. Predicate is consistent. 59ms.
```

This second instance is thought to be a little bit more complicated version of the previous world. It is forced to have two TrafficTickets, two Users, one Authority, one ParkingViolation and only one street. The result is a graph which is great in connections. However, it correctly describes the world as it is designed in our paper.



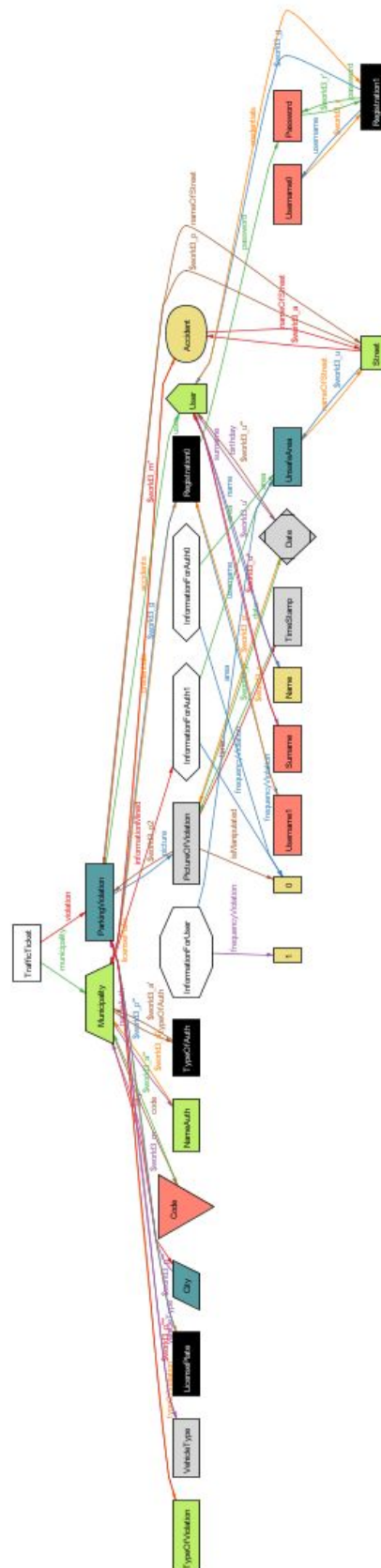
Instance 3

```
pred world3{  
  #GeneralUser = 2  
  #TrafficTicket = 1  
}  
  
run world3 for 3
```

Executing "Run world3 for 3"

```
Solver=sat4j Bitwidth=4 MaxSeq=3 SkolemDepth=1 Symmetry=20  
9462 vars. 822 primary vars. 15445 clauses. 63ms.  
Instance found. Predicate is consistent. 85ms.
```

This instance proves that the presence of a TrafficTicket leads to the presence of at least one Municipality and one User due to our constraints. This is because we forced to have two GeneralUsers and one TrafficTicket. Even if the GeneralUser can be implemented by a User or a Municipality, the only compliant solution is the one with both the two instances.



5. Effort spent

Matteo Ferrini

Description of the Task	Hours
Introduction	3,5
Product perspective	1,5
Product functions	1
Domain assumptions	1,5
External interface Requirements	3
Functional Requirements	1
Non--functional requirements	2
Formal Analysis using Alloy	12

Francesco Gianchino

Description of the Task	Hours
Introduction	2.5
Product perspective	6
Product functions	0.5
Domain assumptions	4
External interface Requirements	0.5
Functional Requirements	6
Non--functional requirements	2
Formal Analysis using Alloy	15

Riccardo Colella

Description of the Task	Hours
Introduction	8
Product perspective	0
Product functions	6
Domain assumptions	1
External interface Requirements	2
Functional Requirements	6
Non--functional requirements	4
Formal Analysis using Alloy	12

6. References

The following documents have been taken into consideration for the RASD writing:

1. IEEE Std 830-1998 IEEE Recommended Practice for Software Requirements Specifications
2. Deriving Specifications from Requirements: an Example, Jackson-Zave
3. General Data Protection Regulation (GDPR)
4. Unified Modeling Language User Guide, Addison-Wesley